MODUL 02 · BAHASA DART

Fondasi Bahasa Pemrograman Dart Modern (Dart 3)

Belajar Dart dari nol secara berurutan: program pertama, variabel dan tipe data, operator, control flow, fungsi, koleksi, null safety, OOP, fitur modern Dart 3 (records, patterns, sealed class), error handling, sampai pemrograman asinkron dengan Future dan Stream. Ditutup dengan studi kasus sistem nilai dan IPK mahasiswa.

Durasi Pertemuan 2 (150 Menit) + Belajar Mandiri

Prasyarat Modul 01: Setup & Flutter Basics

Capaian Pembelajaran

- Menulis dan menjalankan program Dart murni dengan dart run maupun DartPad.
- Memakai variabel, tipe data, final/const/late, interpolasi string, dan konversi tipe dengan benar.
- Menggunakan operator (aritmatika, logika, null-aware, cascade, ternary) dan control flow termasuk switch expression Dart 3.
- Menulis fungsi dengan parameter posisi, named, required, dan default, serta memanfaatkan closure dan higher-order function.
- Mengolah List, Set, dan Map dengan map, where, fold, sort, spread, collection-if, dan collection-for.
- Menerapkan Sound Null Safety untuk mencegah error null saat runtime.
- Merancang class dengan berbagai constructor, getter/setter, inheritance, abstract class, interface, mixin, extension, dan enhanced enum.
- Memakai records, pattern matching, destructuring, dan sealed class dengan exhaustive switch.
- Menangani error dengan try/catch/finally, throw, dan exception kustom.
- Menulis kode asinkron dengan Future, async/await, Future.wait, Stream, async*, dan StreamController.

Daftar Isi Modul

1. Program Dart Pertama: main(), print & Komentar
2. Variabel, Tipe Data & Konversi Tipe
3. Operator: Aritmatika, Logika, Null-aware & Cascade
4. Control Flow: Percabangan & Perulangan
5. Fungsi: Parameter, Closure & Higher-Order Function
6. Koleksi: List, Set & Map

7. Sound Null Safety
8. Object-Oriented Programming (OOP)
9. Fitur Modern Dart 3: Records, Patterns & Sealed Class
10. Error Handling: try, catch & Exception Kustom
11. Pemrograman Asinkron: Future, async/await & Stream
12. Studi Kasus: Sistem Nilai & IPK Mahasiswa
13. Tugas Praktikum
14. Kuis Pemahaman & Kunci Jawaban

1. Program Dart Pertama: main(), print & Komentar

Sebelum menulis aplikasi Flutter, kita perlu akrab dulu dengan bahasa yang menggerakkannya: **Dart**. Di modul ini semua contoh adalah program Dart murni (tanpa Flutter), jadi bisa dijalankan langsung di terminal atau di DartPad.

Di mana menjalankan kode Dart?

- **DartPad** (`dartpad.dev`): cukup buka browser, tempel kode, klik Run. Cocok untuk mencoba potongan kode kecil.
- **Terminal**: Dart SDK sudah ikut terpasang bersama Flutter (Modul 01). Simpan kode sebagai file `.dart`, lalu jalankan dengan `dart run`.

```
# Cek versi Dart (harus 3.x)
dart --version

# Cara 1: jalankan satu file
dart run halo.dart

# Cara 2: buat proyek console lengkap
dart create -t console belajar_dart
cd belajar_dart
dart run
```

Program pertama

Setiap program Dart dimulai dari fungsi `main()`. Saat program dijalankan, Dart mencari fungsi ini dan mengeksekusi isinya baris demi baris dari atas ke bawah.

`halo.dart`

```
void main() {
  print('Halo, Dart!');
  print('Selamat datang di Praktikum Pemrograman Mobile');
}

// Output:
// Halo, Dart!
// Selamat datang di Praktikum Pemrograman Mobile
```

- `void` artinya fungsi `main` tidak mengembalikan nilai apa pun.
- `print()` menampilkan teks ke konsol, lalu pindah ke baris baru.
- Setiap pernyataan (*statement*) diakhiri titik koma `;`. Lupa titik koma adalah error paling sering bagi pemula.
- Teks (String) ditulis di antara tanda kutip, boleh kutip satu `'...'` atau kutip dua `"..."`.

> ℹ️ **Tentang komentar `// Output:`**
>
> Di seluruh modul ini, hasil yang muncul di konsol ditulis sebagai komentar `// Output: ...`. Komentar tidak ikut dijalankan, jadi kode tetap bisa langsung disalin dan dijalankan.

Komentar

Komentar adalah catatan untuk manusia dan diabaikan oleh compiler. Dart punya tiga jenis komentar:

```dart
// Komentar satu baris: diabaikan oleh compiler.

/*
  Komentar banyak baris.
  Cocok untuk menonaktifkan sementara beberapa baris kode.
*/

/// Komentar dokumentasi (tiga garis miring).
/// Muncul sebagai tooltip saat kursor diarahkan ke nama fungsi di VS Code.
void sapa(String nama) {
  print('Halo, $nama!');
}

void main() {
  sapa('Rani'); // Output: Halo, Rani!
  // sapa('Budi'); baris ini tidak dijalankan karena dikomentari
}
```

Gunakan `///` di atas class atau fungsi yang penting. Editor akan menampilkannya sebagai dokumentasi, sama seperti dokumentasi widget Flutter yang muncul saat Anda mengarahkan kursor ke `Text` atau `Column`.

Struktur file Dart

Sebuah file Dart biasanya berisi (berurutan): `import`, deklarasi tingkat atas (variabel, fungsi, class), dan fungsi `main()`. Urutan fungsi dan class di dalam file tidak berpengaruh, karena Dart membaca seluruh file terlebih dahulu.

```dart
import 'dart:math'; // 1. import library bawaan (opsional)

const namaKampus = 'UNSOED'; // 2. variabel tingkat atas (top-level)

int kuadrat(int x) {
  // 3. fungsi tingkat atas
  return x * x;
}

void main() {
  // 4. titik masuk program
  print(namaKampus); // Output: UNSOED
```

```
  print(kuadrat(7)); // Output: 49
  print(max(3, 9)); // Output: 9
}
```

Fungsi `max()` tidak kita tulis sendiri, melainkan berasal dari library `dart:math` yang di-`import` di baris pertama. Library bawaan lain yang sering dipakai: `dart:async` (Future dan Stream), `dart:convert` (JSON), dan `dart:io` (file dan terminal, tidak tersedia di web).

Argumen dari terminal

`main()` boleh menerima parameter `List<String>` berisi argumen yang diketik setelah nama file. Di Flutter, bentuk ini jarang dipakai, tetapi berguna untuk program console.

```dart
void main(List<String> args) {
  print('Jumlah argumen: ${args.length}');
  print('Isi argumen: $args');
}

// Output:
// Jumlah argumen: 0
// Isi argumen: []
```

Jika dijalankan dengan `dart run argumen.dart halo dunia`, hasilnya menjadi `Jumlah argumen: 2` dan `Isi argumen: [halo, dunia]`.

> ⚠️ **Kesalahan umum pemula**
>
> - Lupa titik koma `;` di akhir statement.
> - Menulis `Main()` atau `Print()`. Dart membedakan huruf besar dan kecil (*case-sensitive*).
> - Kurung kurawal `{ }` tidak berpasangan. Gunakan fitur *Format Document* (`Shift+Alt+F`) di VS Code agar indentasi rapi dan kesalahan mudah terlihat.

2. Variabel, Tipe Data & Konversi Tipe

Variabel adalah "kotak bernama" untuk menyimpan data. Dart adalah bahasa yang *strongly typed* (setiap nilai punya tipe yang jelas), tetapi punya *type inference* sehingga kita sering tidak perlu menulis tipenya secara manual.

Tipe data dasar

Tipe	Isi	Contoh
`int`	Bilangan bulat	`int sks = 3;`
`double`	Bilangan desimal	`double ipk = 3.75;`
`num`	Bisa `int` atau `double`	`num nilai = 80;`
`String`	Teks	`String nama = 'Ayu';`
`bool`	`true` atau `false`	`bool aktif = true;`
`dynamic`	Tipe bebas, pemeriksaan tipe dimatikan	`dynamic data = 1;`

Deklarasi dengan tipe eksplisit dan `var`

```dart
void main() {
  // Tipe ditulis eksplisit
  String nama = 'Ahmad Fikri';
  int semester = 5;
  double ipk = 3.84;
  bool aktif = true;

  // Tipe ditebak otomatis dari nilai awal (type inference)
  var jurusan = 'Informatika'; // dianggap String
  var angkatan = 2023; // dianggap int

  print(nama); // Output: Ahmad Fikri
  print(semester); // Output: 5
  print(ipk); // Output: 3.84
  print(aktif); // Output: true
  print(jurusan.runtimeType); // Output: String
  print(angkatan.runtimeType); // Output: int

  semester = 6; // variabel biasa boleh diubah nilainya
  print(semester); // Output: 6

  // jurusan = 123; // ERROR: jurusan sudah bertipe String
}
```

`var` **bukan** berarti "tipe bebas". Begitu diisi `'Informatika'`, tipe `jurusan` terkunci menjadi `String`. Properti `runtimeType` dipakai untuk melihat tipe sebuah nilai saat program berjalan.

Angka: `int`, `double`, dan `num`

```
void main() {
  int sks = 3;
  double nilai = 87.5;
  num apaSaja = 10; // num bisa menampung int maupun double
  apaSaja = 10.5;

  print(sks + 1); // Output: 4
  print(nilai / 2); // Output: 43.75
  print(apaSaja); // Output: 10.5

  double rata = 7; // literal int otomatis diubah menjadi double
  print(rata); // Output: 7.0
  print(1e3); // Output: 1000.0
  print(0xFF); // Output: 255
  print(1000000 == 1_000_000); // Output: true
}
```

Garis bawah pada `1_000_000` hanya pemisah agar angka besar mudah dibaca (fitur Dart 3.6 ke atas). Nilainya sama persis dengan `1000000`.

> ℹ️ **Perbedaan output di DartPad**
>
> DartPad menjalankan kode sebagai JavaScript, sehingga `double` yang nilainya bulat dicetak tanpa `.0` (misalnya `7`, bukan `7.0`). Output di modul ini mengikuti `dart run` di terminal.

String: kutip, escape, multi-baris, raw

```
void main() {
  String a = 'kutip satu';
  String b = "kutip dua";
  String c = "Jum'at"; // kutip satu di dalam kutip dua
  String d = 'Jum\'at'; // atau di-escape dengan backslash

  String multi = '''
Baris pertama
Baris kedua''';

  String raw = r'C:\folder\baru'; // raw string: backslash tidak diproses

  print(a); // Output: kutip satu
  print(b); // Output: kutip dua
  print(c); // Output: Jum'at
  print(d); // Output: Jum'at
  print(multi);
```

```
  // Output:
  // Baris pertama
  // Baris kedua
  print(raw); // Output: C:\folder\baru
  print('Baris 1\nBaris 2');
  // Output:
  // Baris 1
  // Baris 2
}
```

`\n` adalah karakter baris baru dan `\t` adalah tab. Awalan `r` membuat *raw string* sehingga backslash dicetak apa adanya, berguna untuk path Windows dan pola `RegExp`.

String interpolation

Untuk menyisipkan nilai ke dalam teks, gunakan `$nama` untuk variabel sederhana dan `${ekspresi}` untuk ekspresi (perhitungan, pemanggilan method, akses properti).

```
void main() {
  String nama = 'Nadia';
  int sks = 21;
  double ipk = 3.756;

  print('Nama: $nama'); // Output: Nama: Nadia
  print('SKS semester depan: ${sks + 3}'); // Output: SKS semester depan: 24
  print('Panjang nama: ${nama.length} huruf'); // Output: Panjang nama: 5 huruf
  print('IPK: ${ipk.toStringAsFixed(2)}'); // Output: IPK: 3.76
  print('Huruf besar: ${nama.toUpperCase()}'); // Output: Huruf besar: NADIA

  // Menyambung dengan + juga bisa, tetapi interpolasi lebih disarankan
  print('Halo, ' + nama + '!'); // Output: Halo, Nadia!
}
```

> ⚠️ **Lupa kurung kurawal**
>
> `'$nama.length'` akan mencetak `Nadia.length`, bukan `5`, karena tanpa `{ }` Dart hanya mengambil nama variabelnya. Tulis `'${nama.length}'`.

Method String yang sering dipakai

```
void main() {
  String nim = '  H1D023001  ';
  String bersih = nim.trim(); // buang spasi di awal dan akhir

  print(bersih); // Output: H1D023001
  print(bersih.length); // Output: 9
  print(bersih.toLowerCase()); // Output: h1d023001
  print(bersih.startsWith('H1D')); // Output: true
  print(bersih.contains('023')); // Output: true
  print(bersih.substring(3, 5)); // Output: 02
```

```dart
  print(bersih[0]); // Output: H
  print(bersih.replaceAll('0', 'o')); // Output: H1Do23oo1
  print('Flutter,Dart,Supabase'.split(',')); // Output: [Flutter, Dart, Supabase]
  print('7'.padLeft(3, '0')); // Output: 007
  print('ab' * 3); // Output: abababab
  print(''.isEmpty); // Output: true
}
```

`substring(3, 5)` mengambil karakter dari indeks 3 sampai **sebelum** indeks 5. Indeks String selalu dimulai dari 0.

`final`, `const`, dan `late`

```dart
void main() {
  final String namaKampus = 'UNSOED';
  final waktuMulai = DateTime.now(); // nilainya baru diketahui saat program berjalan
  const double nilaiMaksimal = 4.0;
  const jumlahJamSeminggu = 7 * 24; // ekspresi konstan, dihitung saat kompilasi

  // namaKampus = 'UGM'; // ERROR: final hanya bisa diisi sekali
  // nilaiMaksimal = 5.0; // ERROR: const tidak bisa diubah
  // const sekarang = DateTime.now(); // ERROR: DateTime.now() bukan nilai compile-time

  print(namaKampus); // Output: UNSOED
  print(nilaiMaksimal); // Output: 4.0
  print(jumlahJamSeminggu); // Output: 168
  print(waktuMulai.year > 2000); // Output: true
}
```

Kata kunci	Bisa diubah?	Kapan nilainya ditentukan?	Contoh pemakaian
`var` / tipe	Ya	Kapan saja	Penghitung, input pengguna
`final`	Tidak (sekali isi)	Saat program berjalan (runtime)	Data dari API, waktu login
`const`	Tidak	Saat kompilasi (compile-time)	Angka tetap, widget statis Flutter
`late`	Tergantung	Ditunda, diisi sebelum dipakai	Controller di `initState`

`final` mengunci *variabelnya*, bukan isi objeknya. List yang `final` masih bisa ditambah isinya, sedangkan list `const` benar-benar beku:

```dart
void main() {
  final daftarFinal = ['Dart', 'Flutter'];
  daftarFinal.add('Supabase'); // boleh: isi list bisa berubah
  print(daftarFinal); // Output: [Dart, Flutter, Supabase]
  // daftarFinal = []; // ERROR: variabelnya tidak bisa diganti

  const daftarConst = ['Dart', 'Flutter'];
  print(daftarConst.length); // Output: 2
```

```
  // daftarConst.add('Hive'); // ERROR saat runtime: Unsupported operation
}
```

`late` menandai variabel yang **pasti** diisi nanti sebelum dibaca. `late final` juga bisa dipakai untuk inisialisasi malas (*lazy*): nilai baru dihitung saat pertama kali dipakai.

```
late String token; // akan diisi oleh fungsi login()

void login() {
  token = 'abc123';
}

int hitungBerat() {
  print('Menghitung...');
  return 42;
}

void main() {
  login();
  print('Token: $token'); // Output: Token: abc123

  late final int hasil = hitungBerat(); // belum dihitung di baris ini
  print('Sebelum memakai hasil');
  print(hasil); // baru dihitung di sini

  // Output:
  // Sebelum memakai hasil
  // Menghitung...
  // 42
}
```

`var` vs `dynamic` vs `Object`

```
void main() {
  var a = 10; // tipe terkunci: int
  dynamic b = 10; // tipe boleh berganti, tanpa pemeriksaan compiler
  Object c = 10; // menampung apa saja, tetapi tetap diperiksa compiler

  b = 'sekarang String';
  c = true;
  // a = 'teks'; // ERROR: a bertipe int

  print(a); // Output: 10
  print(b); // Output: sekarang String
  print(c); // Output: true

  // print(c.length); // ERROR compile: Object tidak punya length
  // print(b.panjangSekali); // lolos compile, tetapi CRASH saat runtime
}
```

Hindari `dynamic` kecuali benar-benar perlu (misalnya data JSON mentah). Dengan `dynamic`, salah ketik nama properti baru ketahuan ketika aplikasi sudah berjalan.

Konversi tipe

```dart
void main() {
  // String -> angka
  int umur = int.parse('20');
  double ipk = double.parse('3.75');
  print(umur + 1); // Output: 21
  print(ipk); // Output: 3.75

  // tryParse mengembalikan null jika gagal (tidak crash)
  int? gagal = int.tryParse('dua puluh');
  print(gagal); // Output: null

  // angka -> String
  String teks = 42.toString();
  print(teks + '!'); // Output: 42!
  print(3.14159.toStringAsFixed(2)); // Output: 3.14

  // double -> int (toInt memotong, round membulatkan)
  print(9.7.toInt()); // Output: 9
  print(9.7.round()); // Output: 10
  print(9.2.ceil()); // Output: 10
  print(9.7.floor()); // Output: 9

  // int -> double
  print(5.toDouble()); // Output: 5.0
}
```

Gunakan `int.tryParse()` untuk input dari pengguna (misalnya dari `TextField` di Modul 05), karena pengguna bisa saja mengetik huruf. `int.parse()` akan melempar `FormatException` jika teksnya bukan angka.

> 💡 **Ringkasan**
>
> - Pakai `final` secara default. Ganti ke `var` hanya jika nilainya memang perlu berubah.
> - Pakai `const` untuk nilai yang sudah pasti sejak kode ditulis.
> - Gunakan interpolasi `'$x'` / `'${ekspresi}'` untuk menyusun teks.
> - Selalu pakai `tryParse` untuk input yang belum tentu valid.

3. Operator: Aritmatika, Logika, Null-aware & Cascade

Operator adalah simbol untuk mengolah nilai: menjumlah, membandingkan, menggabungkan kondisi, sampai menangani nilai `null`. Banyak operator Dart sama dengan bahasa C, Java, atau JavaScript, tetapi ada beberapa yang khas Dart seperti `~/`, `??`, dan cascade `..`.

Operator aritmatika

```dart
void main() {
  int a = 17;
  int b = 5;

  print(a + b); // Output: 22
  print(a - b); // Output: 12
  print(a * b); // Output: 85
  print(a / b); // Output: 3.4
  print(a ~/ b); // Output: 3
  print(a % b); // Output: 2
  print(-a); // Output: -17
}
```

- `/` selalu menghasilkan `double`, walaupun kedua angkanya `int`.
- `~/` adalah pembagian bulat (hasil dibulatkan ke bawah menuju nol).
- `%` adalah sisa bagi (modulo), sering dipakai untuk mengecek genap/ganjil: `n % 2 == 0`.

Operator penugasan dan increment

```dart
void main() {
  int poin = 10;
  poin += 5; // sama dengan: poin = poin + 5
  print(poin); // Output: 15
  poin -= 3;
  print(poin); // Output: 12
  poin *= 2;
  print(poin); // Output: 24
  poin ~/= 5;
  print(poin); // Output: 4

  int x = 1;
  print(x++); // Output: 1
  print(x); // Output: 2
  print(++x); // Output: 3
  x--;
  print(x); // Output: 2
}
```

`x++` (postfix) memberikan nilai *lama* lalu menambah, sedangkan `++x` (prefix) menambah dulu lalu memberikan nilai *baru*. Itulah sebabnya `print(x++)` mencetak 1.

Operator perbandingan dan uji tipe

```dart
void main() {
  int nilai = 75;
  print(nilai == 75); // Output: true
  print(nilai != 80); // Output: true
  print(nilai > 80); // Output: false
  print(nilai <= 75); // Output: true

  Object data = 'Dart';
  print(data is String); // Output: true
  print(data is! int); // Output: true
  String teks = data as String; // cast: "anggap data sebagai String"
  print(teks.length); // Output: 4
}
```

`is` mengecek tipe, `is!` kebalikannya, dan `as` melakukan *cast*. Jika cast salah (misalnya `data as int`), program akan crash, jadi biasakan mengecek dengan `is` terlebih dahulu.

Operator logika

```dart
void main() {
  int kehadiran = 85; // persen
  double nilaiAkhir = 72.5;
  bool sudahBayarUkt = true;

  bool bolehUjian = kehadiran >= 75 && sudahBayarUkt;
  bool dapatBeasiswa = nilaiAkhir >= 85 || kehadiran == 100;

  print(bolehUjian); // Output: true
  print(dapatBeasiswa); // Output: false
  print(!sudahBayarUkt); // Output: false
}
```

Operator	Nama	Hasil `true` jika...
`a && b`	AND	keduanya `true`
`a || b`	OR	salah satu `true`
`!a`	NOT	`a` bernilai `false`

Dart memakai *short-circuit evaluation*: pada `a && b`, jika `a` sudah `false`, maka `b` tidak dievaluasi sama sekali. Ini berguna untuk pengecekan aman seperti `teks != null && teks.isNotEmpty`.

Operator ternary

Bentuk singkat dari if-else yang menghasilkan nilai: `kondisi ? nilaiJikaBenar : nilaiJikaSalah`. Di Flutter, ternary sangat sering dipakai di dalam widget, misalnya `isLoading ? CircularProgressIndicator() : Text('Data')`.

```dart
void main() {
  int nilai = 68;
  String status = nilai >= 60 ? 'Lulus' : 'Tidak lulus';
  print(status); // Output: Lulus

  int jumlahPesan = 1;
  print('$jumlahPesan pesan${jumlahPesan > 1 ? ' baru' : ''}'); // Output: 1 pesan
}
```

Operator null-aware: `??`, `?.`, `??=`

Operator ini menangani nilai yang mungkin `null`. Pembahasan lengkapnya ada di section Null Safety; berikut perkenalan singkatnya.

```dart
String? cariNamaPanggilan(String nama) {
  // Mengembalikan null jika nama tidak dikenal
  return nama == 'Muhammad Fikri' ? 'Fikri' : null;
}

void main() {
  String? panggilan = cariNamaPanggilan('Siti Aminah'); // null

  print(panggilan ?? 'Belum diisi'); // Output: Belum diisi
  print(panggilan?.length); // Output: null

  panggilan ??= 'Siti'; // isi hanya jika masih null
  print(panggilan); // Output: Siti

  String? lain = cariNamaPanggilan('Muhammad Fikri');
  lain ??= 'Default'; // tidak berubah karena sudah ada isinya
  print(lain); // Output: Fikri
}
```

- `a ?? b`: pakai `a`, tetapi jika `a` null pakai `b`.
- `a?.b`: akses `b` hanya jika `a` tidak null; jika null, hasilnya `null` (tidak crash).
- `a ??= b`: isi `a` dengan `b` hanya jika `a` masih null.

Cascade `..`

Cascade memungkinkan beberapa operasi berurutan pada **objek yang sama** tanpa menulis nama variabelnya berulang kali. Contoh ini memakai class sederhana; class dibahas detail di section OOP.

```dart
class Pesanan {
  String menu = '';
  int jumlah = 0;
  List<String> catatan = [];
```

```dart
  void tampilkan() => print('$jumlah x $menu, catatan: $catatan');
}

void main() {
  // Tanpa cascade
  var p1 = Pesanan();
  p1.menu = 'Mendoan';
  p1.jumlah = 5;
  p1.tampilkan(); // Output: 5 x Mendoan, catatan: []

  // Dengan cascade
  var p2 = Pesanan()
    ..menu = 'Soto Sokaraja'
    ..jumlah = 2
    ..catatan.add('pedas')
    ..tampilkan(); // Output: 2 x Soto Sokaraja, catatan: [pedas]
  print(p2.jumlah); // Output: 2

  // Cascade juga praktis untuk List
  final angka = <int>[]
    ..add(3)
    ..add(1)
    ..add(2)
    ..sort();
  print(angka); // Output: [1, 2, 3]
}
```

Perhatikan: `..add(3)` mengembalikan **objek list-nya**, bukan hasil dari `add`. Karena itu rangkaian di atas tetap menghasilkan list. Ada juga `?..` untuk objek yang mungkin null.

> ⚠️ **Kesalahan umum**
>
> - Mengira `7 / 2` menghasilkan `3`. Hasilnya `3.5`. Gunakan `7 ~/ 2` untuk mendapatkan `3`.
> - Menulis `=` (penugasan) padahal maksudnya `==` (perbandingan).
> - Membandingkan `double` hasil perhitungan dengan `==`: `0.1 + 0.2 == 0.3` bernilai `false` karena keterbatasan presisi bilangan desimal.

4. Control Flow: Percabangan & Perulangan

Control flow menentukan *baris mana yang dijalankan* dan *berapa kali*. Ada dua kelompok besar: percabangan (`if` , `switch`) dan perulangan (`for` , `while`).

if, else if, else

```dart
void main() {
  double nilai = 78.5;

  if (nilai >= 85) {
    print('A');
  } else if (nilai >= 70) {
    print('B');
  } else if (nilai >= 55) {
    print('C');
  } else {
    print('D/E');
  }
  // Output: B
}
```

Kondisi diperiksa dari atas ke bawah, dan hanya blok **pertama** yang kondisinya benar yang dijalankan. Nilai 78.5 tidak memenuhi `>= 85` , tetapi memenuhi `>= 70` , sehingga pengecekan berhenti di situ. Kondisi di Dart harus bertipe `bool` ; menulis `if (1)` seperti di C adalah error.

switch statement

```dart
void main() {
  String hari = 'Sabtu';

  switch (hari) {
    case 'Sabtu':
    case 'Minggu':
      print('Libur kuliah');
    case 'Jumat':
      print('Kuliah sampai jam 11');
    default:
      print('Kuliah seperti biasa');
  }
  // Output: Libur kuliah
}
```

Sejak Dart 3, `break` tidak wajib lagi: case yang berisi kode otomatis berhenti di akhir case-nya. Case yang kosong (seperti `'Sabtu'`) "jatuh" ke case berikutnya, sehingga Sabtu dan Minggu berbagi kode yang sama.

switch expression (Dart 3)

Switch expression adalah switch yang **menghasilkan nilai**. Bentuknya lebih ringkas: tidak ada `case` , tidak ada `break` , setiap cabang ditulis `pola => nilai` , dan `_` berarti "selain itu".

```dart
void main() {
  String nilaiHuruf = 'B';
  int bobot = switch (nilaiHuruf) {
    'A' => 4,
    'B' => 3,
    'C' => 2,
    'D' => 1,
    _ => 0,
  };
  print('Bobot nilai $nilaiHuruf = $bobot'); // Output: Bobot nilai B = 3

  int nomorHari = 6;
  String jenisHari = switch (nomorHari) {
    6 || 7 => 'Akhir pekan',
    >= 1 && <= 5 => 'Hari kerja',
    _ => 'Nomor hari tidak valid',
  };
  print(jenisHari); // Output: Akhir pekan
}
```

`6 || 7` berarti "6 atau 7", dan `>= 1 && <= 5` berarti "di antara 1 sampai 5". Ini adalah *pattern* yang akan dibahas lebih dalam di section Fitur Modern Dart 3.

Perulangan for

```dart
void main() {
  for (int i = 1; i <= 3; i++) {
    print('Pertemuan ke-$i');
  }
  // Output:
  // Pertemuan ke-1
  // Pertemuan ke-2
  // Pertemuan ke-3

  // Perulangan bersarang (nested): tabel perkalian 3 x 3
  for (int baris = 1; baris <= 3; baris++) {
    String isi = '';
    for (int kolom = 1; kolom <= 3; kolom++) {
      isi += '${baris * kolom}'.padLeft(4);
    }
    print(isi);
  }
  // Output:
  //    1   2   3
  //    2   4   6
  //    3   6   9
}
```

Bagian `for (inisialisasi; kondisi; perubahan)` : `i` mulai dari 1, perulangan berjalan selama `i <= 3`, dan setelah tiap putaran `i` bertambah 1.

for-in dan forEach

```dart
void main() {
  List<String> matkul = ['Pemrograman Mobile', 'Basis Data', 'Jaringan Komputer'];

  // for-in: ambil setiap elemen tanpa indeks
  for (final mk in matkul) {
    print('- $mk');
  }
  // Output:
  // - Pemrograman Mobile
  // - Basis Data
  // - Jaringan Komputer

  // for biasa jika butuh nomor urut
  for (int i = 0; i < matkul.length; i++) {
    print('${i + 1}. ${matkul[i]}');
  }
  // Output:
  // 1. Pemrograman Mobile
  // 2. Basis Data
  // 3. Jaringan Komputer

  // forEach dengan fungsi
  matkul.forEach((mk) => print(mk.toUpperCase()));
  // Output:
  // PEMROGRAMAN MOBILE
  // BASIS DATA
  // JARINGAN KOMPUTER
}
```

while dan do-while

```dart
void main() {
  int saldo = 100000;
  int hari = 0;
  while (saldo >= 25000) {
    saldo -= 25000;
    hari++;
  }
  print('Uang jajan cukup untuk $hari hari, sisa $saldo'); // Output: Uang jajan cukup untuk 4 hari,
sisa 0

  int percobaan = 0;
  do {
    percobaan++;
    print('Percobaan login ke-$percobaan');
  } while (percobaan < 2);
  // Output:
  // Percobaan login ke-1
  // Percobaan login ke-2
}
```

`while` mengecek kondisi **sebelum** menjalankan isi perulangan, sedangkan `do-while` menjalankan isi minimal satu kali, baru mengecek kondisi di akhir. Gunakan `while` jika jumlah

putaran tidak diketahui sejak awal.

break dan continue

```
void main() {
  List<int> daftarNilai = [80, 45, 90, -1, 70];

  for (final n in daftarNilai) {
    if (n < 0) {
      print('Data tidak valid, berhenti.');
      break; // keluar dari perulangan sepenuhnya
    }
    if (n < 60) {
      print('$n dilewati');
      continue; // lompat ke elemen berikutnya
    }
    print('$n lulus');
  }
  // Output:
  // 80 lulus
  // 45 dilewati
  // 90 lulus
  // Data tidak valid, berhenti.
}
```

Nilai 70 tidak pernah diproses karena `break` sudah menghentikan perulangan saat bertemu -1.

> ⚠️ **Kesalahan umum**
>
> - **Infinite loop**: lupa mengubah variabel kondisi di dalam `while` (misalnya lupa `hari++`), sehingga program tidak pernah berhenti.
> - **Off-by-one**: memakai `i <= list.length` padahal indeks terakhir adalah `length - 1`. Akibatnya muncul `RangeError`.
> - Lupa cabang `_` pada switch expression untuk tipe `int` / `String`. Compiler akan menolak karena tidak semua kemungkinan tertangani.

5. Fungsi: Parameter, Closure & Higher-Order Function

Fungsi membungkus sekumpulan perintah agar bisa dipakai berulang kali dengan nama. Di Flutter, fungsi ada di mana-mana: `build()`, `onPressed`, `itemBuilder`, semuanya adalah fungsi. Memahami parameter bernama dan fungsi sebagai nilai adalah kunci membaca kode Flutter.

Fungsi dasar: parameter dan nilai kembali

```
double hitungLuas(double panjang, double lebar) {
  return panjang * lebar;
}

void sapa(String nama) {
  print('Halo, $nama!');
}

void main() {
  double luas = hitungLuas(4, 2.5);
  print(luas); // Output: 10.0
  sapa('Dimas'); // Output: Halo, Dimas!
}
```

Format umum: `TipeKembali namaFungsi(TipeParam param) { ... }`. Fungsi `void` tidak mengembalikan nilai. Fungsi yang punya tipe kembali wajib mengakhiri setiap jalurnya dengan `return`.

Arrow function `=>`

Jika isi fungsi hanya satu ekspresi, tulis dengan panah. `=> ekspresi` sama dengan `{ return ekspresi; }`.

```
int kuadrat(int x) => x * x;
bool isGenap(int x) => x % 2 == 0;
String salam(String nama) => 'Selamat pagi, $nama';

// Fungsi rekursif (memanggil dirinya sendiri)
int faktorial(int n) => n <= 1 ? 1 : n * faktorial(n - 1);

void main() {
  print(kuadrat(9)); // Output: 81
  print(isGenap(7)); // Output: false
  print(salam('Ayu')); // Output: Selamat pagi, Ayu
  print(faktorial(5)); // Output: 120
}
```

Parameter posisi opsional `[ ]`

```
String perkenalan(String nama, [String asal = 'Purwokerto', int? umur]) {
  var teks = 'Saya $nama dari $asal';
  if (umur != null) {
    teks += ', umur $umur tahun';
  }
```

```
  return teks;
}

void main() {
  print(perkenalan('Rani')); // Output: Saya Rani dari Purwokerto
  print(perkenalan('Bayu', 'Cilacap')); // Output: Saya Bayu dari Cilacap
  print(perkenalan('Sari', 'Banyumas', 19)); // Output: Saya Sari dari Banyumas, umur 19 tahun
}
```

Parameter di dalam `[ ]` boleh tidak diisi. Isinya bisa diberi nilai default (`asal = 'Purwokerto'`) atau bertipe nullable (`int? umur`) yang otomatis bernilai `null`. Kelemahannya: urutan argumen harus diingat.

Named parameter `{ }`, `required`, dan default

Named parameter dipanggil dengan menyebut namanya, sehingga urutan bebas dan kode mudah dibaca. Gaya inilah yang dipakai hampir semua widget Flutter, misalnya `Text('Halo', style: ..., maxLines: 2)`.

```
void daftarMahasiswa({
  required String nama,
  required String nim,
  String prodi = 'Informatika',
  int? semester,
}) {
  print('$nim - $nama ($prodi), semester: ${semester ?? '-'}');
}

void main() {
  daftarMahasiswa(nama: 'Ayu', nim: 'H1D023010');
  daftarMahasiswa(
    nim: 'H1D023011',
    nama: 'Bima',
    semester: 3,
    prodi: 'Teknik Elektro',
  );
  // daftarMahasiswa(nama: 'Cici'); // ERROR: parameter nim wajib diisi

  // Output:
  // H1D023010 - Ayu (Informatika), semester: -
  // H1D023011 - Bima (Teknik Elektro), semester: 3
}
```

Jenis	Deklarasi	Pemanggilan
Posisi wajib	`f(int a)`	`f(1)`
Posisi opsional	`f([int a = 0])`	`f()` atau `f(1)`
Named opsional	`f({int a = 0})` / `f({int? a})`	`f()` atau `f(a: 1)`
Named wajib	`f({required int a})`	`f(a: 1)`

Parameter posisi dan named boleh digabung, asalkan yang posisi ditulis lebih dulu, misalnya `void kirim(String tujuan, {bool penting = false})`.

Fungsi sebagai nilai dan fungsi anonim

Di Dart, fungsi adalah objek: bisa disimpan di variabel, dikirim sebagai argumen, dan dikembalikan dari fungsi lain. Fungsi tanpa nama disebut **fungsi anonim** (lambda).

```
void main() {
  // Fungsi anonim disimpan dalam variabel bertipe fungsi
  int Function(int, int) tambah = (a, b) => a + b;
  print(tambah(2, 3)); // Output: 5

  // Fungsi anonim dengan body blok
  var cetakBintang = (int n) {
    print('*' * n);
  };
  cetakBintang(5); // Output: *****

  // Fungsi anonim langsung dipakai sebagai argumen
  var angka = [3, 1, 2];
  angka.forEach((n) {
    print('angka $n');
  });
  // Output:
  // angka 3
  // angka 1
  // angka 2
}
```

Bentuk `(n) { ... }` dan `(n) => ...` inilah yang Anda tulis di Flutter, misalnya `onPressed: () { ... }`.

Closure: fungsi yang "mengingat" variabel

```
int Function() buatPenghitung() {
  int hitung = 0;
  return () {
    hitung++;
    return hitung;
```

```
  };
}

void main() {
  var klikTombolA = buatPenghitung();
  var klikTombolB = buatPenghitung();

  print(klikTombolA()); // Output: 1
  print(klikTombolA()); // Output: 2
  print(klikTombolA()); // Output: 3
  print(klikTombolB()); // Output: 1
}
```

Fungsi yang dikembalikan tetap bisa mengakses variabel `hitung` walaupun `buatPenghitung()` sudah selesai dijalankan. Setiap pemanggilan `buatPenghitung()` membuat variabel `hitung` sendiri, sehingga penghitung A dan B tidak saling mengganggu.

Higher-order function

Fungsi yang menerima fungsi lain sebagai parameter, atau mengembalikan fungsi, disebut *higher-order function*.

```
int hitung(int a, int b, int Function(int, int) operasi) {
  return operasi(a, b);
}

int Function(int) pengali(int faktor) {
  return (x) => x * faktor;
}

int kuadrat(int x) => x * x;

void main() {
  print(hitung(10, 4, (a, b) => a + b)); // Output: 14
  print(hitung(10, 4, (a, b) => a * b)); // Output: 40

  final kaliDua = pengali(2);
  final kaliTiga = pengali(3);
  print(kaliDua(5)); // Output: 10
  print(kaliTiga(5)); // Output: 15

  // Mengirim fungsi yang sudah ada tanpa tanda kurung (tear-off)
  print([1, 2, 3].map(kuadrat).toList()); // Output: [1, 4, 9]
}
```

`typedef` dan callback

`typedef` memberi nama untuk tipe fungsi agar mudah dibaca. Flutter memakai pola ini, misalnya `VoidCallback` dan `ValueChanged<T>`.

```
typedef SaatDitekan = void Function(String namaTombol);

void buatTombol(String label, {required SaatDitekan onPressed}) {
  print('[Tombol $label dibuat]');
  onPressed(label); // simulasi pengguna menekan tombol
```

```dart
}

void main() {
  buatTombol('Simpan', onPressed: (nama) {
    print('Tombol $nama ditekan, data disimpan.');
  });
  // Output:
  // [Tombol Simpan dibuat]
  // Tombol Simpan ditekan, data disimpan.
}
```

Pola ini sama persis dengan `ElevatedButton(onPressed: () { ... })`: kita "menitipkan" fungsi, lalu widget yang memanggilnya ketika tombol benar-benar ditekan.

> ⚠️ **Kesalahan umum**
>
> - `onPressed: simpan()` memanggil fungsi *saat itu juga* dan mengirim hasilnya. Yang benar `onPressed: simpan` (tear-off) atau `onPressed: () => simpan()`.
> - Lupa `return` pada fungsi blok yang punya tipe kembali.
> - Menulis arrow function dengan blok: `() => { ... }`. Di Dart, ini bukan blok, melainkan Set literal. Gunakan `() { ... }`.

6. Koleksi: List, Set & Map

Koleksi menyimpan banyak nilai dalam satu variabel. Dart punya tiga koleksi utama: **List** (berurutan, boleh duplikat), **Set** (unik), dan **Map** (pasangan kunci-nilai). Hampir semua data di aplikasi, mulai dari daftar produk sampai respon JSON dari API, disimpan dalam koleksi.

List: dasar-dasar

```
void main() {
  List<String> buah = ['apel', 'jeruk'];
  buah.add('mangga'); // tambah di akhir
  buah.addAll(['salak', 'durian']); // tambah banyak sekaligus
  buah.insert(0, 'pisang'); // sisipkan di indeks 0

  print(buah); // Output: [pisang, apel, jeruk, mangga, salak, durian]
  print(buah.length); // Output: 6
  print(buah[1]); // Output: apel
  print(buah.first); // Output: pisang
  print(buah.last); // Output: durian
  print(buah.indexOf('mangga')); // Output: 3
  print(buah.contains('nanas')); // Output: false

  buah[1] = 'anggur'; // ubah elemen
  buah.remove('jeruk'); // hapus berdasarkan nilai
  buah.removeAt(0); // hapus berdasarkan indeks
  print(buah); // Output: [anggur, mangga, salak, durian]
  print(buah.sublist(1, 3)); // Output: [mangga, salak]
  print(buah.reversed.toList()); // Output: [durian, salak, mangga, anggur]
  print(buah.isEmpty); // Output: false
}
```

Indeks List dimulai dari 0. Mengakses indeks di luar jangkauan (misalnya `buah[10]`) menyebabkan `RangeError`. Tipe di dalam `< >` disebut *generic*: `List<String>` berarti list yang isinya hanya boleh String.

Membuat List secara otomatis

```
void main() {
  var kuadrat = List.generate(5, (i) => i * i);
  print(kuadrat); // Output: [0, 1, 4, 9, 16]

  var kursi = List.filled(3, 'kosong');
  print(kursi); // Output: [kosong, kosong, kosong]

  var kosong = <int>[]; // list kosong bertipe int
  print(kosong.length); // Output: 0
}
```

Set: koleksi tanpa duplikat

```
void main() {
  Set<String> hobi = {'coding', 'futsal', 'membaca'};
```

```
  hobi.add('coding'); // diabaikan karena sudah ada
  print(hobi); // Output: {coding, futsal, membaca}
  print(hobi.length); // Output: 3
  print(hobi.contains('membaca')); // Output: true

  var kelasA = {'Ayu', 'Bima', 'Citra'};
  var kelasB = {'Bima', 'Dodi', 'Citra'};
  print(kelasA.union(kelasB)); // Output: {Ayu, Bima, Citra, Dodi}
  print(kelasA.intersection(kelasB)); // Output: {Bima, Citra}
  print(kelasA.difference(kelasB)); // Output: {Ayu}

  // Trik populer: buang duplikat dari List
  var presensi = ['01', '02', '01', '03', '02'];
  print(presensi.toSet().toList()); // Output: [01, 02, 03]
}
```

> ⚠️ **`{}` kosong adalah Map**
>
> `var x = {};` menghasilkan `Map`, bukan `Set`. Untuk Set kosong tulis `<String>{}` atau `Set<String> x = {};`.

Map: pasangan kunci-nilai

```
void main() {
  Map<String, int> stok = {
    'pulpen': 20,
    'buku': 15,
  };

  stok['penggaris'] = 8; // tambah pasangan baru
  stok['buku'] = 12; // ubah nilai
  print(stok); // Output: {pulpen: 20, buku: 12, penggaris: 8}
  print(stok['pulpen']); // Output: 20
  print(stok['penghapus']); // Output: null
  print(stok.containsKey('buku')); // Output: true
  print(stok.keys); // Output: (pulpen, buku, penggaris)
  print(stok.values.toList()); // Output: [20, 12, 8]

  stok.remove('pulpen');
  stok.update('buku', (lama) => lama + 5);
  stok.putIfAbsent('spidol', () => 10);
  print(stok); // Output: {buku: 17, penggaris: 8, spidol: 10}

  for (final entry in stok.entries) {
    print('${entry.key} -> ${entry.value}');
  }
  // Output:
  // buku -> 17
  // penggaris -> 8
  // spidol -> 10
}
```

Mengakses kunci yang tidak ada menghasilkan `null`, bukan error. Karena itu tipe `stok['x']` adalah `int?` (nullable).

Map<String, dynamic>: bentuk data JSON

Data JSON dari API (Modul 09) akan berbentuk `Map<String, dynamic>` karena nilainya bisa bermacam-macam tipe.

```
void main() {
  Map<String, dynamic> mahasiswa = {
    'nama': 'Nadia',
    'nim': 'H1D023005',
    'ipk': 3.75,
    'aktif': true,
    'matkul': ['Mobile', 'Web'],
  };

  String nama = mahasiswa['nama'] as String;
  double ipk = mahasiswa['ipk'] as double;
  List matkul = mahasiswa['matkul'] as List;

  print('$nama, IPK $ipk, ambil ${matkul.length} matkul'); // Output: Nadia, IPK 3.75, ambil 2
matkul
}
```

Method fungsional: map, where, fold, reduce, any, every

Method berikut menerima fungsi sebagai argumen (ingat higher-order function di section sebelumnya) dan membuat kode pengolahan data jauh lebih ringkas daripada perulangan manual.

```
void main() {
  List<int> nilai = [78, 92, 55, 64, 88, 40];

  var lulus = nilai.where((n) => n >= 60).toList();
  print(lulus); // Output: [78, 92, 64, 88]

  var plusBonus = nilai.map((n) => n + 5).toList();
  print(plusBonus); // Output: [83, 97, 60, 69, 93, 45]

  int total = nilai.fold(0, (jumlah, n) => jumlah + n);
  print(total); // Output: 417
  print(total / nilai.length); // Output: 69.5

  int tertinggi = nilai.reduce((a, b) => a > b ? a : b);
  print(tertinggi); // Output: 92

  print(nilai.any((n) => n < 50)); // Output: true
  print(nilai.every((n) => n >= 40)); // Output: true
  print(nilai.firstWhere((n) => n > 80)); // Output: 92
  print(nilai.take(2).toList()); // Output: [78, 92]
  print(nilai.skip(4).toList()); // Output: [88, 40]
  print(nilai.join(' | ')); // Output: 78 | 92 | 55 | 64 | 88 | 40
}
```

Method	Fungsi	Hasil
`where`	Menyaring elemen yang memenuhi syarat	Iterable
`map`	Mengubah setiap elemen	Iterable
`fold`	Mengakumulasi dengan nilai awal	Satu nilai
`reduce`	Mengakumulasi tanpa nilai awal (list tidak boleh kosong)	Satu nilai
`any` / `every`	Ada yang memenuhi / semua memenuhi	`bool`
`firstWhere`	Elemen pertama yang memenuhi syarat	Satu elemen

`where` dan `map` menghasilkan `Iterable` yang bersifat *lazy* (baru diproses ketika dibutuhkan). Tambahkan `.toList()` jika Anda butuh List sungguhan.

Mengurutkan dengan sort

```dart
void main() {
  var angka = [5, 2, 9, 1];
  angka.sort(); // naik (ascending)
  print(angka); // Output: [1, 2, 5, 9]
  angka.sort((a, b) => b.compareTo(a)); // turun (descending)
  print(angka); // Output: [9, 5, 2, 1]

  var nama = ['Citra', 'ayu', 'Bima'];
  nama.sort(); // huruf besar diurutkan lebih dulu
  print(nama); // Output: [Bima, Citra, ayu]
  nama.sort((a, b) => a.toLowerCase().compareTo(b.toLowerCase()));
  print(nama); // Output: [ayu, Bima, Citra]

  var mhs = [
    {'nama': 'Ayu', 'ipk': 3.4},
    {'nama': 'Bima', 'ipk': 3.9},
    {'nama': 'Citra', 'ipk': 3.6},
  ];
  mhs.sort((a, b) => (b['ipk'] as double).compareTo(a['ipk'] as double));
  print(mhs.map((m) => m['nama']).toList()); // Output: [Bima, Citra, Ayu]
}
```

Fungsi pembanding mengembalikan angka negatif jika `a` harus di depan, positif jika `b` di depan, dan 0 jika setara. `compareTo` sudah melakukan itu untuk kita. Perhatikan bahwa `sort()` mengubah list aslinya dan tidak mengembalikan nilai.

Spread, collection-if, dan collection-for

```dart
List<String>? ambilMatkulTambahan() => null; // anggap belum ada data
```

```dart
void main() {
  var wajib = ['Algoritma', 'Basis Data'];
  var pilihan = ['Pemrograman Mobile'];
  var tambahan = ambilMatkulTambahan();
  bool ambilMagang = true;

  var krs = [
    ...wajib, // sebar isi list wajib
    ...pilihan,
    ...?tambahan, // sebar hanya jika tidak null
    if (ambilMagang) 'Magang Industri',
    for (var i = 1; i <= 2; i++) 'Seminar $i',
  ];
  print(krs);
  // Output:
  // [Algoritma, Basis Data, Pemrograman Mobile, Magang Industri, Seminar 1, Seminar 2]

  var nilai = {'Ayu': 90, 'Bima': 55};
  var label = [
    for (final e in nilai.entries) '${e.key}: ${e.value >= 60 ? 'Lulus' : 'Gagal'}',
  ];
  print(label); // Output: [Ayu: Lulus, Bima: Gagal]
}
```

Fitur ini sangat sering dipakai di Flutter untuk menyusun `children` sebuah `Column`, misalnya `children: [const Header(), if (isLogin) const Profil(), ...daftarMenu]`.

Merangkai method (chaining)

```dart
void main() {
  final belanja = [
    {'nama': 'Kopi', 'harga': 15000, 'kategori': 'minuman'},
    {'nama': 'Mendoan', 'harga': 10000, 'kategori': 'makanan'},
    {'nama': 'Es Teh', 'harga': 5000, 'kategori': 'minuman'},
    {'nama': 'Nasi Rames', 'harga': 18000, 'kategori': 'makanan'},
  ];

  final totalMinuman = belanja
      .where((b) => b['kategori'] == 'minuman')
      .map((b) => b['harga'] as int)
      .fold(0, (a, b) => a + b);
  print('Total minuman: Rp$totalMinuman'); // Output: Total minuman: Rp20000

  final namaMahal = belanja
      .where((b) => (b['harga'] as int) >= 15000)
      .map((b) => (b['nama'] as String).toUpperCase())
      .toList();
  print(namaMahal); // Output: [KOPI, NASI RAMES]
}
```

Baca rantai dari atas ke bawah: saring yang kategori minuman, ambil harganya, lalu jumlahkan. Gaya ini lebih mudah dibaca daripada `for` dengan banyak variabel sementara.

⚠️ **Kesalahan umum**

- Lupa `.toList()` setelah `map` / `where`, lalu bingung kenapa tercetak `(...)` bukan `[...]`.
- Memanggil `reduce` atau `first` pada list kosong, sehingga muncul `StateError: No element`. Gunakan `fold` atau cek `isEmpty` dulu.
- Menulis `var hasil = list.sort();`. `sort()` bertipe `void`, list aslinya yang terurut.

7. Sound Null Safety

Di banyak bahasa, error paling sering adalah memakai nilai yang ternyata kosong (*null*), dikenal sebagai `NullPointerException`. Dart 3 menerapkan **Sound Null Safety**: secara default, semua tipe **tidak boleh** null, dan compiler memaksa kita menangani kemungkinan null sebelum program dijalankan.

Tipe non-nullable vs nullable

```dart
void main() {
  String nama = 'Ayu';
  // nama = null; // ERROR: String tidak boleh null

  String? nomorHp; // tanda ? artinya boleh null, nilai awalnya null
  print(nama.length); // Output: 3
  print(nomorHp); // Output: null
  // print(nomorHp.length); // ERROR: nomorHp mungkin null

  nomorHp = '08123456789';
  print(nomorHp.length); // Output: 11
}
```

Setelah `nomorHp` diisi String, Dart tahu nilainya pasti tidak null, sehingga `nomorHp.length` diizinkan. Kemampuan ini disebut **type promotion**.

Ringkasan operator null safety

Sintaks	Arti	Contoh
`Tipe?`	Tipe nullable (boleh null)	`String? email;`
`?.`	Akses hanya jika tidak null	`email?.length`
`??`	Nilai cadangan jika null	`email ?? '-'`
`??=`	Isi hanya jika masih null	`email ??= 'a@b.c';`
`!`	Paksa anggap tidak null (crash jika ternyata null)	`email!.length`
`late`	Diisi nanti, dijanjikan sebelum dipakai	`late String token;`
`required`	Named parameter wajib diisi	`{required String nim}`

Memakai `?.`, `??`, dan `??=`

```dart
String? cariEmail(String nim) {
  final data = {'H1D023001': 'ayu@mhs.unsoed.ac.id'};
  return data[nim]; // null jika nim tidak terdaftar
}

void main() {
  String? email1 = cariEmail('H1D023001');
  String? email2 = cariEmail('H1D099999');

  print(email1?.toUpperCase()); // Output: AYU@MHS.UNSOED.AC.ID
  print(email2?.toUpperCase()); // Output: null
  print(email2 ?? 'Email belum terdaftar'); // Output: Email belum terdaftar
  print(email2?.length ?? 0); // Output: 0

  email2 ??= 'default@unsoed.ac.id';
  print(email2); // Output: default@unsoed.ac.id
}
```

`email2?.length ?? 0` adalah kombinasi yang sangat umum: "ambil panjangnya jika ada, jika tidak pakai 0".

Type promotion dengan pengecekan null

```dart
int hitungPanjang(String? teks) {
  if (teks == null) {
    return 0;
  }
  // Di sini Dart tahu teks pasti tidak null
  return teks.length;
}

class Profil {
  String? bio;
}

void cetakBio(Profil p) {
  // if (p.bio != null) print(p.bio.length); // ERROR: field tidak dipromosikan
  final bio = p.bio; // salin ke variabel lokal dulu
  if (bio != null) {
    print('Panjang bio: ${bio.length}');
  } else {
    print('Bio kosong');
  }
}

void main() {
  print(hitungPanjang(null)); // Output: 0
  print(hitungPanjang('Flutter')); // Output: 7

  cetakBio(Profil()); // Output: Bio kosong
  cetakBio(Profil()..bio = 'Suka Dart'); // Output: Panjang bio: 9
}
```

Promotion berjalan untuk variabel lokal dan parameter. Untuk field class yang publik dan bisa diubah, Dart tidak bisa menjamin nilainya tetap sama di antara pengecekan dan pemakaian, sehingga trik "salin ke variabel lokal" sangat sering dipakai.

Operator `!`: gunakan dengan hati-hati

```dart
void main() {
  Map<String, int> nilai = {'Ayu': 90};

  int nilaiAyu = nilai['Ayu']!; // kita YAKIN kunci 'Ayu' ada
  print(nilaiAyu); // Output: 90

  // int nilaiBudi = nilai['Budi']!;
  // CRASH saat runtime: Null check operator used on a null value

  int nilaiBudi = nilai['Budi'] ?? 0; // cara yang aman
  print(nilaiBudi); // Output: 0
}
```

`!` mematikan perlindungan null safety. Gunakan hanya jika Anda benar-benar yakin nilainya tidak null. Dalam banyak kasus, `??` atau pengecekan `if` adalah pilihan yang lebih aman.

`late` pada field class

```dart
class KoneksiDatabase {
  late String namaDb; // belum ada nilai saat objek dibuat

  void buka(String nama) {
    namaDb = nama;
    print('Database $namaDb dibuka');
  }
}

void main() {
  final db = KoneksiDatabase();
  // print(db.namaDb); // CRASH: LateInitializationError, belum diisi
  db.buka('kampus.db'); // Output: Database kampus.db dibuka
  print(db.namaDb); // Output: kampus.db
}
```

Di Flutter, `late` biasa dipakai untuk controller yang dibuat di `initState()`, misalnya `late TextEditingController _namaController;`.

Null-aware pada List dan Map

```dart
List<String>? ambilDaftar(bool ada) => ada ? ['Dart', 'Flutter'] : null;

void main() {
  final daftar = ambilDaftar(false);
  print(daftar?.length); // Output: null
  print(daftar?[0]); // Output: null
  print(daftar?.isEmpty ?? true); // Output: true
```

```
  final daftar2 = ambilDaftar(true);
  print(daftar2?[1]); // Output: Flutter
}
```

⚠️ **Kesalahan umum**

- Menaburkan `!` di mana-mana agar error compiler hilang. Error-nya hanya pindah menjadi crash saat aplikasi dipakai.
- Membuat semua variabel nullable (`String?`) "supaya aman". Justru sebaliknya: jadikan non-nullable sebisa mungkin, dan nullable hanya untuk data yang memang bisa tidak ada.
- Memakai `late` lalu lupa mengisinya, sehingga muncul `LateInitializationError`.

8. Object-Oriented Programming (OOP)

Flutter sepenuhnya berbasis Object-Oriented Programming (OOP): setiap widget adalah objek dari sebuah class. Section ini membahas OOP di Dart secara bertahap, mulai dari class sederhana sampai mixin, extension, dan enum.

1. Class dan objek

Class adalah cetakan, **objek** adalah hasil cetakannya. Class berisi *field* (data) dan *method* (perilaku).

```
class Mahasiswa {
  String nama;
  String nim;
  int semester;

  // Constructor singkat: langsung mengisi field
  Mahasiswa(this.nama, this.nim, this.semester);

  void perkenalan() {
    print('Halo, saya $nama ($nim), semester $semester.');
  }

  void naikSemester() {
    semester++;
  }
}

void main() {
  var ayu = Mahasiswa('Ayu', 'H1D023001', 3);
  var bima = Mahasiswa('Bima', 'H1D023002', 5);

  ayu.perkenalan();
  ayu.naikSemester();
  ayu.perkenalan();
  bima.perkenalan();
  // Output:
  // Halo, saya Ayu (H1D023001), semester 3.
  // Halo, saya Ayu (H1D023001), semester 4.
  // Halo, saya Bima (H1D023002), semester 5.
}
```

Kata kunci `new` tidak diperlukan di Dart modern. `ayu` dan `bima` adalah dua objek terpisah: mengubah semester `ayu` tidak memengaruhi `bima`.

2. Variasi constructor

```
class Buku {
  final String judul;
  final String penulis;
  final int tahun;
  final int stok;
```

```dart
  // a. Constructor utama dengan named parameter
  Buku({
    required this.judul,
    required this.penulis,
    this.tahun = 2024,
    this.stok = 0,
  });

  // b. Named constructor: cara lain membuat objek
  Buku.ebook(this.judul, this.penulis)
      : tahun = 2025,
        stok = 999;

  // c. Redirecting constructor: meneruskan ke constructor utama
  Buku.anonim(String judul) : this(judul: judul, penulis: 'Anonim');

  @override
  String toString() => '$judul oleh $penulis ($tahun), stok $stok';
}

void main() {
  print(Buku(judul: 'Dart Dasar', penulis: 'Rina', stok: 5));
  print(Buku.ebook('Flutter Cepat', 'Andi'));
  print(Buku.anonim('Catatan Kuliah'));
  // Output:
  // Dart Dasar oleh Rina (2024), stok 5
  // Flutter Cepat oleh Andi (2025), stok 999
  // Catatan Kuliah oleh Anonim (2024), stok 0
}
```

Method `toString()` di-*override* agar `print(objek)` menampilkan teks yang informatif, bukan `Instance of 'Buku'`. Named constructor sering ditemui di Flutter, misalnya `ListView.builder` dan `EdgeInsets.all`.

3. Initializer list dan assert

Bagian setelah tanda `:` pada constructor disebut *initializer list*. Bagian ini dijalankan sebelum body constructor dan cocok untuk mengisi field `final` hasil perhitungan.

```dart
class Mahasiswa {
  final String nim;
  final String nama;
  final String angkatan;

  Mahasiswa(this.nim, String nama)
      : assert(nim.length == 9, 'NIM harus 9 karakter'),
        nama = nama.trim(),
        angkatan = '20${nim.substring(4, 6)}';
}

void main() {
  final m = Mahasiswa('H1D023001', '  Ayu Lestari ');
  print('[${m.nama}]'); // Output: [Ayu Lestari]
  print(m.angkatan); // Output: 2023
}
```

`assert` hanya aktif saat mode debug (Flutter `flutter run`, atau `dart run --enable-asserts`). Jika kondisinya salah, program berhenti dengan pesan yang diberikan.

4. Const constructor

```dart
class Warna {
  final int r, g, b;
  const Warna(this.r, this.g, this.b);
}

void main() {
  const merah1 = Warna(255, 0, 0);
  const merah2 = Warna(255, 0, 0);
  print(identical(merah1, merah2)); // Output: true

  final biru1 = Warna(0, 0, 255);
  final biru2 = Warna(0, 0, 255);
  print(identical(biru1, biru2)); // Output: false
}
```

Objek `const` dengan isi yang sama hanya dibuat **sekali** dan dipakai bersama (`identical` bernilai `true`). Inilah alasan Flutter menyarankan `const Text('Halo')`: widget tidak perlu dibuat ulang setiap kali layar di-*rebuild*. Syaratnya, semua field harus `final`.

5. Factory constructor

`factory` adalah constructor yang boleh mengandung logika dan mengembalikan objek (baru atau yang sudah ada). Pemakaian paling umum adalah `fromJson` untuk mengubah data JSON menjadi objek.

```dart
class Mahasiswa {
  final String nim;
  final String nama;
  final double ipk;

  Mahasiswa({required this.nim, required this.nama, required this.ipk});

  factory Mahasiswa.fromJson(Map<String, dynamic> json) {
    return Mahasiswa(
      nim: json['nim'] as String,
      nama: json['nama'] as String,
      ipk: (json['ipk'] as num).toDouble(),
    );
  }

  Map<String, dynamic> toJson() => {'nim': nim, 'nama': nama, 'ipk': ipk};
}

void main() {
  final json = {'nim': 'H1D023007', 'nama': 'Nadia', 'ipk': 4};
  final m = Mahasiswa.fromJson(json);
  print('${m.nama} - IPK ${m.ipk}'); // Output: Nadia - IPK 4.0
  print(m.toJson()); // Output: {nim: H1D023007, nama: Nadia, ipk: 4.0}
}
```

Perhatikan `(json['ipk'] as num).toDouble()`: server bisa mengirim `4` (int) atau `3.5` (double). Membaca sebagai `num` lalu mengubah ke `double` menghindari error tipe. Pola ini akan dipakai lagi di Modul 09 (REST API).

Factory juga bisa mengembalikan objek yang sama terus-menerus (pola *singleton*):

```
class Pengaturan {
  static final Pengaturan _instance = Pengaturan._internal();
  String tema = 'terang';

  Pengaturan._internal(); // constructor private

  factory Pengaturan() => _instance;
}

void main() {
  final a = Pengaturan();
  final b = Pengaturan();
  a.tema = 'gelap';
  print(b.tema); // Output: gelap
  print(identical(a, b)); // Output: true
}
```

6. Getter, setter, dan private

Nama yang diawali garis bawah `_` bersifat **private**. Akses dari luar diatur lewat *getter* (`get`) dan *setter* (`set`), sehingga data bisa divalidasi.

```
class Mahasiswa {
  final String nama;
  double _ipk = 0;

  Mahasiswa(this.nama);

  double get ipk => _ipk;

  set ipk(double nilai) {
    if (nilai < 0 || nilai > 4) {
      print('IPK $nilai tidak valid, diabaikan.');
      return;
    }
    _ipk = nilai;
  }

  // Getter hasil perhitungan (computed property)
  String get predikat => _ipk >= 3.51 ? 'Cumlaude' : 'Reguler';
}

void main() {
  final m = Mahasiswa('Fikri');
  m.ipk = 3.8; // memanggil setter
  print(m.ipk); // Output: 3.8
  print(m.predikat); // Output: Cumlaude
  m.ipk = 4.5; // Output: IPK 4.5 tidak valid, diabaikan.
```

```
  print(m.ipk); // Output: 3.8
}
```

Contoh lain yang lebih lengkap: rekening dengan saldo dan riwayat yang tidak bisa diubah sembarangan dari luar.

```
class RekeningBank {
  final String pemilik;
  double _saldo = 0;
  final List<String> _riwayat = [];

  RekeningBank(this.pemilik);

  double get saldo => _saldo;
  List<String> get riwayat => List.unmodifiable(_riwayat);

  void setor(double jumlah) {
    if (jumlah <= 0) return;
    _saldo += jumlah;
    _catat('Setor $jumlah');
  }

  bool tarik(double jumlah) {
    if (jumlah > _saldo) {
      _catat('Gagal tarik $jumlah');
      return false;
    }
    _saldo -= jumlah;
    _catat('Tarik $jumlah');
    return true;
  }

  void _catat(String aktivitas) => _riwayat.add(aktivitas); // method private
}

void main() {
  final rek = RekeningBank('Ayu');
  rek.setor(50000);
  rek.tarik(20000);
  rek.tarik(100000);
  print(rek.saldo); // Output: 30000.0
  print(rek.riwayat); // Output: [Setor 50000.0, Tarik 20000.0, Gagal tarik 100000.0]
}
```

> ℹ️ **Private di Dart = private per file**
>
> Dart tidak punya kata kunci `private` / `public`. Awalan `_` membuat anggota hanya terlihat di **file (library) yang sama**. Jadi pada contoh satu file di atas, `main()` secara teknis masih bisa mengakses `_saldo`. Saat class dipindah ke file lain (misalnya `lib/models/rekening.dart`), barulah `_saldo` benar-benar tersembunyi.

7. Anggota static

```dart
class Mahasiswa {
  static int jumlahDibuat = 0; // milik class, bukan milik objek
  static const String kampus = 'UNSOED';

  final String nama;

  Mahasiswa(this.nama) {
    jumlahDibuat++;
  }

  static String formatNim(String angkatan, int urut) {
    return 'H1D$angkatan${urut.toString().padLeft(3, '0')}';
  }
}

void main() {
  final daftar = [Mahasiswa('Ayu'), Mahasiswa('Bima')];
  print(daftar.length); // Output: 2
  print(Mahasiswa.jumlahDibuat); // Output: 2
  print(Mahasiswa.kampus); // Output: UNSOED
  print(Mahasiswa.formatNim('023', 7)); // Output: H1D023007
}
```

Anggota `static` dipanggil lewat nama class, tanpa membuat objek. Contoh di Flutter: `Colors.blue` dan `Navigator.push(...)`.

8. Inheritance (pewarisan) dan polymorphism

```dart
class Pengguna {
  final String nama;
  final String email;

  Pengguna(this.nama, this.email);

  String get peran => 'Pengguna';

  void tampilkanProfil() {
    print('[$peran] $nama <$email>');
  }
}

class Mahasiswa extends Pengguna {
  final String nim;

  // super parameter: diteruskan langsung ke constructor Pengguna
  Mahasiswa(super.nama, super.email, this.nim);

  @override
  String get peran => 'Mahasiswa';

  @override
  void tampilkanProfil() {
    super.tampilkanProfil(); // jalankan versi induk dulu
    print('  NIM: $nim');
```

```
  }
}

class Dosen extends Pengguna {
  final String nidn;

  // cara lama: memanggil super(...) di initializer list
  Dosen(String nama, String email, this.nidn) : super(nama, email);

  @override
  String get peran => 'Dosen';
}

void main() {
  final daftar = <Pengguna>[
    Mahasiswa('Ayu', 'ayu@mhs.unsoed.ac.id', 'H1D023001'),
    Dosen('Dr. Budi', 'budi@unsoed.ac.id', '0612345678'),
  ];

  for (final p in daftar) {
    p.tampilkanProfil();
  }
  // Output:
  // [Mahasiswa] Ayu <ayu@mhs.unsoed.ac.id>
  //   NIM: H1D023001
  // [Dosen] Dr. Budi <budi@unsoed.ac.id>
}
```

`extends` mewarisi seluruh field dan method induk. `@override` menandai method yang ditulis ulang. Walaupun list bertipe `List<Pengguna>`, setiap objek menjalankan versi method miliknya sendiri. Inilah **polymorphism**. Di Flutter, `class HomePage extends StatelessWidget` adalah contoh nyata inheritance.

9. Abstract class

Abstract class adalah class "setengah jadi" yang tidak bisa dibuat objeknya. Ia mendefinisikan method yang **wajib** diisi oleh class turunannya.

```
import 'dart:math';

abstract class BangunDatar {
  String get nama;
  double luas(); // method abstrak: tanpa isi
  double keliling();

  // method biasa boleh ada di abstract class
  void info() {
    print('$nama: luas ${luas().toStringAsFixed(2)}, '
        'keliling ${keliling().toStringAsFixed(2)}');
  }
}

class Lingkaran extends BangunDatar {
  final double r;
  Lingkaran(this.r);
```

```dart
  @override
  String get nama => 'Lingkaran';
  @override
  double luas() => pi * r * r;
  @override
  double keliling() => 2 * pi * r;
}

class PersegiPanjang extends BangunDatar {
  final double p, l;
  PersegiPanjang(this.p, this.l);

  @override
  String get nama => 'Persegi panjang';
  @override
  double luas() => p * l;
  @override
  double keliling() => 2 * (p + l);
}

void main() {
  // final b = BangunDatar(); // ERROR: abstract class tidak bisa diinstansiasi
  final daftar = [Lingkaran(7), PersegiPanjang(4, 3)];
  for (final b in daftar) {
    b.info();
  }
  // Output:
  // Lingkaran: luas 153.94, keliling 43.98
  // Persegi panjang: luas 12.00, keliling 14.00
}
```

10. Interface dengan `implements`

Setiap class di Dart otomatis juga merupakan *interface*. Dengan `implements`, sebuah class berjanji menyediakan **semua** method milik class lain, tetapi tidak mewarisi isinya. Sebuah class boleh meng-`implements` beberapa interface sekaligus.

```dart
abstract class BisaDicetak {
  void cetak();
}

abstract class BisaDiekspor {
  String keCsv();
}

class Transkrip implements BisaDicetak, BisaDiekspor {
  final String nama;
  final Map<String, String> nilai;

  Transkrip(this.nama, this.nilai);

  @override
  void cetak() {
    print('Transkrip $nama');
    nilai.forEach((mk, huruf) => print('  $mk: $huruf'));
  }
```

```
  @override
  String keCsv() => nilai.entries.map((e) => '${e.key},${e.value}').join(';');
}

void main() {
  final t = Transkrip('Ayu', {'Mobile': 'A', 'Basis Data': 'B'});
  t.cetak();
  print(t.keCsv());
  // Output:
  // Transkrip Ayu
  //   Mobile: A
  //   Basis Data: B
  // Mobile,A;Basis Data,B
}
```

	`extends`	`implements`	`with` (mixin)
Jumlah	Hanya satu induk	Boleh banyak	Boleh banyak
Mewarisi isi method?	Ya	Tidak, wajib tulis ulang semua	Ya
Hubungan	"adalah" (Mahasiswa adalah Pengguna)	"berjanji bisa"	"punya kemampuan"

11. Mixin

Mixin adalah kumpulan kemampuan yang bisa "ditempelkan" ke banyak class tanpa harus menjadi induknya. Kata kunci `on` membatasi mixin hanya untuk turunan class tertentu, sehingga mixin boleh memakai anggota class tersebut.

```
class Pengguna {
  final String nama;
  Pengguna(this.nama);
}

mixin AksesPerpustakaan {
  int batasPinjam = 3;
  final List<String> _pinjaman = [];

  void pinjamBuku(String judul) {
    if (_pinjaman.length >= batasPinjam) {
      print('Gagal: batas pinjam $batasPinjam buku');
      return;
    }
    _pinjaman.add(judul);
    print('Meminjam "$judul" (${_pinjaman.length}/$batasPinjam)');
  }
}

mixin AksesLab on Pengguna {
  void masukLab(String lab) => print('$nama masuk $lab'); // boleh pakai nama
```

```dart
}

class Mahasiswa extends Pengguna with AksesPerpustakaan, AksesLab {
  Mahasiswa(super.nama);
}

class Dosen extends Pengguna with AksesPerpustakaan {
  Dosen(super.nama) {
    batasPinjam = 10;
  }
}

void main() {
  final m = Mahasiswa('Ayu');
  m.pinjamBuku('Clean Code');
  m.pinjamBuku('Dart Apprentice');
  m.pinjamBuku('Flutter in Action');
  m.pinjamBuku('Refactoring');
  m.masukLab('Lab Komputasi');

  final d = Dosen('Pak Budi');
  d.pinjamBuku('Design Patterns');
  // Output:
  // Meminjam "Clean Code" (1/3)
  // Meminjam "Dart Apprentice" (2/3)
  // Meminjam "Flutter in Action" (3/3)
  // Gagal: batas pinjam 3 buku
  // Ayu masuk Lab Komputasi
  // Meminjam "Design Patterns" (1/10)
}
```

Anda akan menemui mixin di Flutter, misalnya `class _HomeState extends State<Home> with SingleTickerProviderStateMixin` untuk animasi.

12. Extension method

Extension menambahkan method baru ke class yang sudah ada (bahkan class bawaan seperti `int` dan `String`) tanpa mengubah atau mewarisinya.

```dart
extension FormatRupiah on int {
  String get rupiah {
    final s = toString();
    final buffer = StringBuffer();
    for (var i = 0; i < s.length; i++) {
      if (i > 0 && (s.length - i) % 3 == 0) buffer.write('.');
      buffer.write(s[i]);
    }
    return 'Rp$buffer';
  }
}

extension StringKampus on String {
  bool get isNimValid => RegExp(r'^H1D\d{6}$').hasMatch(this);

  String get kapital =>
      isEmpty ? this : this[0].toUpperCase() + substring(1).toLowerCase();
}
```

```dart
void main() {
  print(150000.rupiah); // Output: Rp150.000
  print(1250000.rupiah); // Output: Rp1.250.000
  print('H1D023001'.isNimValid); // Output: true
  print('abc'.isNimValid); // Output: false
  print('fLUTTER'.kapital); // Output: Flutter
}
```

13. Enum dan enhanced enum

Enum mewakili sekumpulan nilai tetap. Kombinasi enum dengan `switch` membuat compiler bisa memastikan semua kemungkinan sudah ditangani.

```dart
enum StatusKrs { draft, diajukan, disetujui, ditolak }

void main() {
  var status = StatusKrs.diajukan;
  print(status); // Output: StatusKrs.diajukan
  print(status.name); // Output: diajukan
  print(status.index); // Output: 1
  print(StatusKrs.values.length); // Output: 4

  final pesan = switch (status) {
    StatusKrs.draft => 'Belum diajukan',
    StatusKrs.diajukan => 'Menunggu dosen wali',
    StatusKrs.disetujui => 'KRS disetujui',
    StatusKrs.ditolak => 'Perlu revisi',
  };
  print(pesan); // Output: Menunggu dosen wali
}
```

Switch di atas tidak butuh `_` karena semua nilai enum sudah ditulis. Jika nanti ditambahkan nilai baru, misalnya `StatusKrs.dibatalkan`, compiler langsung menunjukkan switch mana yang belum lengkap.

Enhanced enum (Dart 2.17+) bisa punya field, constructor, getter, dan method:

```dart
enum NilaiHuruf {
  a('A', 4.0, 85),
  ab('AB', 3.5, 80),
  b('B', 3.0, 70),
  bc('BC', 2.5, 65),
  c('C', 2.0, 60),
  d('D', 1.0, 50),
  e('E', 0.0, 0);

  const NilaiHuruf(this.label, this.bobot, this.batasBawah);

  final String label;
  final double bobot;
  final int batasBawah;

  bool get lulus => bobot >= 2.0;
```

```
  static NilaiHuruf dariAngka(num nilai) {
    return values.firstWhere((h) => nilai >= h.batasBawah);
  }
}

void main() {
  for (final n in [92, 81, 67, 45]) {
    final huruf = NilaiHuruf.dariAngka(n);
    print('$n -> ${huruf.label} (bobot ${huruf.bobot}, lulus: ${huruf.lulus})');
  }
  // Output:
  // 92 -> A (bobot 4.0, lulus: true)
  // 81 -> AB (bobot 3.5, lulus: true)
  // 67 -> BC (bobot 2.5, lulus: true)
  // 45 -> E (bobot 0.0, lulus: false)
}
```

`values` berisi semua nilai enum sesuai urutan penulisan. Karena urutannya dari batas tertinggi, `firstWhere` menemukan huruf pertama yang batas bawahnya terpenuhi. Skala nilai di sini hanya contoh; sesuaikan dengan pedoman akademik yang berlaku.

> ⚠️ **Kesalahan umum OOP**
>
> - Lupa `@override`. Kode tetap jalan, tetapi Anda kehilangan peringatan saat salah ketik nama method induk.
> - Field non-nullable tidak diisi di constructor: error *"Non-nullable instance field must be initialized"*. Isi lewat `this.x`, initializer list, nilai default, atau gunakan `late`/nullable.
> - Membuat semua field publik dan bisa diubah. Biasakan field `final` dan sembunyikan data penting dengan `_`.

9. Fitur Modern Dart 3: Records, Patterns & Sealed Class

Dart 3 menambahkan fitur yang membuat kode lebih ringkas dan lebih aman: **records** (mengelompokkan beberapa nilai tanpa membuat class), **patterns** (membongkar dan mencocokkan bentuk data), serta **sealed class** (switch yang dijamin lengkap oleh compiler).

Records

Record adalah kumpulan beberapa nilai yang dibungkus jadi satu, ditulis dengan tanda kurung. Field-nya bisa berdasarkan posisi (diakses dengan `$1`, `$2`, ...) atau bernama.

```dart
void main() {
  var mhs = ('Ayu', 3.75); // record posisi
  print(mhs.$1); // Output: Ayu
  print(mhs.$2); // Output: 3.75

  ({String nama, int sks}) matkul = (nama: 'Pemrograman Mobile', sks: 3);
  print(matkul.nama); // Output: Pemrograman Mobile
  print(matkul); // Output: (nama: Pemrograman Mobile, sks: 3)

  var campur = ('H1D023001', nama: 'Ayu', aktif: true); // posisi + bernama
  print(campur.$1); // Output: H1D023001
  print(campur.aktif); // Output: true

  // Record dibandingkan berdasarkan isinya
  print(('Ayu', 20) == ('Ayu', 20)); // Output: true
}
```

Fungsi yang mengembalikan beberapa nilai

Sebelum Dart 3, mengembalikan lebih dari satu nilai harus memakai List, Map, atau class khusus. Dengan record, cukup tulis tipe kembalian sebagai record.

```dart
({double rataRata, int tertinggi, int terendah}) hitungStatistik(List<int> nilai) {
  final urut = [...nilai]..sort();
  final total = nilai.fold(0, (a, b) => a + b);
  return (
    rataRata: total / nilai.length,
    tertinggi: urut.last,
    terendah: urut.first,
  );
}

void main() {
  final s = hitungStatistik([80, 65, 90, 75]);
  print('Rata-rata: ${s.rataRata}'); // Output: Rata-rata: 77.5
  print('Tertinggi: ${s.tertinggi}'); // Output: Tertinggi: 90
  print('Terendah: ${s.terendah}'); // Output: Terendah: 65

  // Langsung dibongkar (destructuring) ke variabel
  final (:rataRata, :tertinggi, :terendah) = hitungStatistik([60, 70]);
  print('$rataRata $tertinggi $terendah'); // Output: 65.0 70 60
}
```

`[...nilai]..sort()` menyalin list dulu sebelum diurutkan agar list asli tidak berubah. `:rataRata` adalah singkatan dari `rataRata: rataRata`.

Destructuring

Destructuring berarti membongkar isi record, List, atau Map langsung ke beberapa variabel.

```
void main() {
  var (nama, ipk) = ('Bima', 3.6);
  print('$nama: $ipk'); // Output: Bima: 3.6

  // Menukar dua variabel tanpa variabel bantu
  var a = 1;
  var b = 2;
  (a, b) = (b, a);
  print('a=$a, b=$b'); // Output: a=2, b=1

  // List pattern dengan rest element (...)
  var [pertama, kedua, ...sisa] = [10, 20, 30, 40];
  print('$pertama $kedua $sisa'); // Output: 10 20 [30, 40]

  // Destructuring di for-in: Map entries
  final nilai = {'Ayu': 90, 'Bima': 78};
  for (final MapEntry(:key, :value) in nilai.entries) {
    print('$key mendapat $value');
  }
  // Output:
  // Ayu mendapat 90
  // Bima mendapat 78

  // .indexed menghasilkan record (indeks, elemen)
  for (final (i, mk) in ['Mobile', 'Web'].indexed) {
    print('${i + 1}. $mk');
  }
  // Output:
  // 1. Mobile
  // 2. Web
}
```

Pattern matching di switch

Pattern di `switch` bisa mencocokkan **rentang nilai** (relational pattern), **tipe**, **bentuk List**, dan **record**, ditambah syarat tambahan dengan `when` (guard).

```
String kategoriNilai(int nilai) => switch (nilai) {
      >= 85 && <= 100 => 'Sangat baik',
      >= 70 && < 85 => 'Baik',
      >= 60 && < 70 => 'Cukup',
      >= 0 && < 60 => 'Kurang',
      _ => 'Nilai tidak valid',
    };

void main() {
  print(kategoriNilai(91)); // Output: Sangat baik
  print(kategoriNilai(70)); // Output: Baik
  print(kategoriNilai(59)); // Output: Kurang
```

```
  print(kategoriNilai(120)); // Output: Nilai tidak valid
}
```

```
String jelaskan(Object data) => switch (data) {
      int n when n < 0 => 'Bilangan negatif $n',
      int n => 'Bilangan bulat $n',
      double d => 'Desimal ${d.toStringAsFixed(1)}',
      String s when s.isEmpty => 'Teks kosong',
      String s => 'Teks "$s" (${s.length} huruf)',
      [] => 'List kosong',
      [var x] => 'List berisi satu: $x',
      [var x, ...] => 'List diawali $x',
      (String nama, int umur) => 'Record: $nama umur $umur',
      _ => 'Tipe lain: ${data.runtimeType}',
    };

void main() {
  print(jelaskan(-5)); // Output: Bilangan negatif -5
  print(jelaskan(42)); // Output: Bilangan bulat 42
  print(jelaskan(3.14159)); // Output: Desimal 3.1
  print(jelaskan('')); // Output: Teks kosong
  print(jelaskan('Dart')); // Output: Teks "Dart" (4 huruf)
  print(jelaskan([])); // Output: List kosong
  print(jelaskan([7])); // Output: List berisi satu: 7
  print(jelaskan([1, 2, 3])); // Output: List diawali 1
  print(jelaskan(('Ayu', 20))); // Output: Record: Ayu umur 20
  print(jelaskan(true)); // Output: Tipe lain: bool
}
```

Cabang dicek dari atas ke bawah. `int n when n < 0` artinya "jika data bertipe int, simpan sebagai `n`, *dan* nilainya negatif". Karena itu cabang yang lebih spesifik harus ditulis lebih dulu.

if-case: validasi bentuk data

Pattern juga bisa dipakai di `if` dengan kata kunci `case`. Ini sangat praktis untuk memeriksa JSON: cek kunci yang wajib ada *sekaligus* tipenya, lalu langsung ambil nilainya.

```
void prosesJson(Map<String, dynamic> json) {
  if (json case {'nama': String nama, 'ipk': double ipk}) {
    print('Valid: $nama, IPK $ipk');
  } else {
    print('Format JSON tidak sesuai: $json');
  }
}

void main() {
  prosesJson({'nama': 'Ayu', 'ipk': 3.8}); // Output: Valid: Ayu, IPK 3.8
  prosesJson({'nama': 'Bima'}); // Output: Format JSON tidak sesuai: {nama: Bima}
  prosesJson({'nama': 'Citra', 'ipk': '3.5'}); // Output: Format JSON tidak sesuai: {nama: Citra,
ipk: 3.5}
}
```

Data ketiga ditolak karena `'3.5'` adalah String, bukan double. Tanpa pattern, kita harus menulis beberapa `if` dan `is` secara manual.

Object pattern

Object pattern mencocokkan objek berdasarkan nilai getter/field-nya.

```
class Mahasiswa {
  final String nama;
  final int semester;
  final double ipk;
  const Mahasiswa(this.nama, this.semester, this.ipk);
}

String evaluasi(Mahasiswa m) => switch (m) {
      Mahasiswa(semester: > 14) => '${m.nama}: melewati batas masa studi',
      Mahasiswa(:var nama, ipk: >= 3.5) => '$nama: kandidat cumlaude',
      Mahasiswa(:var nama, :var ipk) when ipk < 2.0 => '$nama: peringatan akademik',
      Mahasiswa(:var nama) => '$nama: normal',
    };

void main() {
  print(evaluasi(const Mahasiswa('Ayu', 5, 3.8))); // Output: Ayu: kandidat cumlaude
  print(evaluasi(const Mahasiswa('Bima', 15, 3.0))); // Output: Bima: melewati batas masa studi
  print(evaluasi(const Mahasiswa('Citra', 4, 1.9))); // Output: Citra: peringatan akademik
  print(evaluasi(const Mahasiswa('Dodi', 6, 3.1))); // Output: Dodi: normal
}
```

Sealed class dan exhaustive switch

`sealed class` adalah abstract class yang turunannya hanya boleh ditulis di file yang sama. Karena compiler tahu **semua** turunannya, switch terhadap sealed class tidak perlu cabang `_`, dan compiler akan memberi error jika ada turunan yang terlewat. Ini disebut *exhaustive checking*.

```
sealed class HasilLogin {}

class LoginBerhasil extends HasilLogin {
  final String nama;
  LoginBerhasil(this.nama);
}

class PasswordSalah extends HasilLogin {
  final int sisaPercobaan;
  PasswordSalah(this.sisaPercobaan);
}

class AkunDiblokir extends HasilLogin {}

HasilLogin login(String username, String password) {
  if (username == 'admin') return AkunDiblokir();
  if (password == 'rahasia') return LoginBerhasil(username);
  return PasswordSalah(2);
}

String pesanUntuk(HasilLogin hasil) => switch (hasil) {
      LoginBerhasil(:final nama) => 'Selamat datang, $nama!',
      PasswordSalah(:final sisaPercobaan) =>
        'Password salah. Sisa percobaan: $sisaPercobaan',
      AkunDiblokir() => 'Akun diblokir, hubungi admin.',
    };
```

```
void main() {
  print(pesanUntuk(login('ayu', 'rahasia'))); // Output: Selamat datang, ayu!
  print(pesanUntuk(login('ayu', '12345'))); // Output: Password salah. Sisa percobaan: 2
  print(pesanUntuk(login('admin', 'apa saja'))); // Output: Akun diblokir, hubungi admin.
}
```

Coba hapus baris `AkunDiblokir() => ...`, lalu jalankan `dart analyze`. Compiler akan menolak dengan pesan bahwa tipe `HasilLogin` tidak tertangani secara lengkap (*not exhaustively matched*).

Pola yang sama sangat cocok untuk status pemuatan data di aplikasi (Memuat, Sukses, Gagal), yang akan Anda pakai bersama controller GetX dan REST API:

```
sealed class StatusData {}

class Memuat extends StatusData {}

class Sukses extends StatusData {
  final List<String> data;
  Sukses(this.data);
}

class Gagal extends StatusData {
  final String pesan;
  Gagal(this.pesan);
}

String tampilan(StatusData status) => switch (status) {
      Memuat() => 'Menampilkan loading...',
      Sukses(data: []) => 'Belum ada data',
      Sukses(:final data) => 'Menampilkan ${data.length} item',
      Gagal(:final pesan) => 'Error: $pesan',
    };

void main() {
  print(tampilan(Memuat())); // Output: Menampilkan loading...
  print(tampilan(Sukses([]))); // Output: Belum ada data
  print(tampilan(Sukses(['Ayu', 'Bima']))); // Output: Menampilkan 2 item
  print(tampilan(Gagal('Koneksi terputus'))); // Output: Error: Koneksi terputus
}
```

Di Flutter, fungsi seperti `tampilan()` akan mengembalikan widget (misalnya `CircularProgressIndicator()` atau `ListView`), bukan String.

Class modifier (ringkas)

Dart 3 menambahkan kata kunci untuk mengatur bagaimana class boleh dipakai **dari file/library lain**. Di dalam file yang sama, batasan ini tidak berlaku.

Modifier	Boleh dibuat objek?	Boleh `extends` ?	Boleh `implements` ?	Kegunaan
`abstract`	Tidak	Ya	Ya	Kerangka dengan method abstrak
`base`	Ya	Ya	Tidak	Wajib mewarisi implementasi asli
`interface`	Ya	Tidak	Ya	Murni kontrak
`final`	Ya	Tidak	Tidak	Class tertutup, tidak bisa diturunkan
`sealed`	Tidak	Hanya di file sama	Hanya di file sama	Exhaustive switch
`mixin class`	Ya	Ya	Ya	Bisa dipakai sebagai class maupun mixin

Kombinasi yang paling sering dipakai di aplikasi adalah `abstract interface class` untuk kontrak *repository*, sehingga sumber data mudah diganti (misalnya data palsu untuk testing di Modul 15):

```dart
abstract interface class MahasiswaRepository {
  List<String> ambilSemuaNama();
}

final class RepositoryMemori implements MahasiswaRepository {
  final _data = ['Ayu', 'Bima', 'Citra'];

  @override
  List<String> ambilSemuaNama() => List.unmodifiable(_data);
}

void tampilkan(MahasiswaRepository repo) {
  print(repo.ambilSemuaNama().join(', '));
}

void main() {
  tampilkan(RepositoryMemori()); // Output: Ayu, Bima, Citra
}
```

💡 **Ringkasan**

- **Record**: kembalikan beberapa nilai sekaligus tanpa membuat class.
- **Destructuring**: `var (a, b) = ...`, `var [x, ...sisa] = ...`, `for (final (i, e) in list.indexed)`.
- **Pattern** di switch dan `if-case`: cocokkan rentang, tipe, dan bentuk data.
- **Sealed class**: kumpulan status yang terbatas, dan compiler memastikan semuanya ditangani.

10. Error Handling: try, catch & Exception Kustom

Program yang baik tidak langsung "mati" ketika terjadi masalah, seperti input yang salah, data rusak, atau server tidak merespons. Dart menyediakan `try`, `catch`, `on`, `finally`, dan `throw` untuk menangani situasi tersebut dengan rapi.

try dan catch

```
void main() {
  try {
    int angka = int.parse('12a'); // melempar FormatException
    print('Angka: $angka'); // baris ini dilewati
  } catch (e) {
    print('Terjadi error: ${e.runtimeType}');
  }
  print('Program tetap berjalan');
  // Output:
  // Terjadi error: FormatException
  // Program tetap berjalan
}
```

Begitu terjadi error di dalam `try`, sisa baris di blok itu dilewati dan eksekusi melompat ke `catch`. Setelah itu, program berlanjut normal. Tanpa `try/catch`, program akan berhenti dengan pesan *Unhandled exception*.

on, catch, finally, dan throw

```
double bagi(int a, int b) {
  if (b == 0) {
    throw ArgumentError('Pembagi tidak boleh nol');
  }
  return a / b;
}

void main() {
  for (final input in ['10', 'x', '0']) {
    try {
      final b = int.parse(input);
      print('100 / $b = ${bagi(100, b)}');
    } on FormatException {
      print('"$input" bukan angka');
    } on ArgumentError catch (e) {
      print('Argumen salah: ${e.message}');
    } finally {
      print('-- selesai memproses "$input"');
    }
  }
  // Output:
  // 100 / 10 = 10.0
  // -- selesai memproses "10"
  // "x" bukan angka
  // -- selesai memproses "x"
  // Argumen salah: Pembagi tidak boleh nol
```

```
  // -- selesai memproses "0"
}
```

- `on TipeError` menangkap jenis error tertentu saja. Tambahkan `catch (e)` jika butuh objek error-nya.
- `finally` **selalu** dijalankan, baik terjadi error maupun tidak. Cocok untuk menutup koneksi atau mematikan indikator loading.
- `throw` melempar error secara sengaja ketika kondisi tidak valid.
- `catch (e, stackTrace)` memberikan parameter kedua berisi jejak pemanggilan, berguna untuk debugging.

Membuat Exception sendiri

Exception kustom membuat jenis error lebih jelas dan bisa membawa data tambahan. Cukup buat class yang meng-`implements Exception`.

```dart
class SaldoTidakCukupException implements Exception {
  final double saldo;
  final double diminta;

  SaldoTidakCukupException(this.saldo, this.diminta);

  @override
  String toString() => 'SaldoTidakCukupException: saldo $saldo, diminta $diminta';
}

class Dompet {
  double saldo;
  Dompet(this.saldo);

  void bayar(double jumlah) {
    if (jumlah <= 0) {
      throw ArgumentError.value(jumlah, 'jumlah', 'harus lebih dari 0');
    }
    if (jumlah > saldo) {
      throw SaldoTidakCukupException(saldo, jumlah);
    }
    saldo -= jumlah;
  }
}

void main() {
  final dompet = Dompet(50000);
  try {
    dompet.bayar(20000);
    print('Bayar 20000 berhasil, sisa ${dompet.saldo}');
    dompet.bayar(40000);
    print('Baris ini tidak dijalankan');
  } on SaldoTidakCukupException catch (e) {
    print(e);
    print('Kurang: ${e.diminta - e.saldo}');
  }
  // Output:
  // Bayar 20000 berhasil, sisa 30000.0
```

```
  // SaldoTidakCukupException: saldo 30000.0, diminta 40000.0
  // Kurang: 10000.0
}
```

rethrow: mencatat lalu meneruskan error

```
int bacaNilai(String input) {
  try {
    return int.parse(input);
  } on FormatException {
    print('[log] input tidak valid: $input');
    rethrow; // lempar ulang ke pemanggil
  }
}

void main() {
  try {
    bacaNilai('sembilan');
  } catch (e) {
    print('Ditangani di main: ${e.runtimeType}');
  }
  // Output:
  // [log] input tidak valid: sembilan
  // Ditangani di main: FormatException
}
```

`rethrow` berguna ketika sebuah fungsi ingin mencatat error (logging) tetapi keputusan penanganannya, misalnya menampilkan pesan ke pengguna, tetap diserahkan ke pemanggil.

Exception vs Error

	`Exception`	`Error`
Arti	Kondisi tidak normal yang **bisa diantisipasi**	**Bug** di dalam kode
Contoh	`FormatException`, `TimeoutException`, gagal koneksi	`RangeError`, `TypeError`, `LateInitializationError`
Sikap	Tangkap dan tangani (tampilkan pesan, coba lagi)	Perbaiki kodenya, jangan ditangkap diam-diam

⚠️ **Kesalahan umum**

- `catch (e) {}` kosong: error "ditelan" sehingga bug sulit dilacak. Minimal cetak atau catat error-nya.
- Menangkap semua error dengan `catch` padahal yang diharapkan hanya satu jenis. Gunakan `on TipeException` agar lebih spesifik.
- Melempar String (`throw 'gagal'`). Hal ini memang diizinkan, tetapi lebih baik melempar objek `Exception` agar bisa ditangkap berdasarkan tipenya.

11. Pemrograman Asinkron: Future, async/await & Stream

Aplikasi mobile sering harus menunggu: mengambil data dari internet, membaca database, atau menunggu input. Jika program "diam menunggu", layar akan membeku. Dart memakai model *single-threaded event loop* dengan `Future` dan `Stream` agar pekerjaan yang lama berjalan tanpa memblokir tampilan.

Kode sinkron vs asinkron

```dart
void main() {
  print('1. Mulai');
  Future.delayed(const Duration(seconds: 1), () {
    print('3. Data selesai diambil (setelah 1 detik)');
  });
  print('2. Lanjut mengerjakan hal lain');
  // Output:
  // 1. Mulai
  // 2. Lanjut mengerjakan hal lain
  // 3. Data selesai diambil (setelah 1 detik)
}
```

`Future.delayed` "menjadwalkan" pekerjaan untuk nanti, lalu Dart langsung lanjut ke baris berikutnya. Itulah sebabnya baris 2 tercetak sebelum baris 3. `Future` adalah janji bahwa sebuah nilai (atau error) akan tersedia **di masa depan**.

async dan await

```dart
Future<String> ambilNamaMahasiswa(String nim) async {
  await Future.delayed(const Duration(milliseconds: 500)); // simulasi jaringan
  return 'Ayu Lestari';
}

Future<void> main() async {
  print('Mengambil data...');
  String nama = await ambilNamaMahasiswa('H1D023001');
  print('Nama: $nama');
  print('Selesai');
  // Output:
  // Mengambil data...
  // Nama: Ayu Lestari
  // Selesai
}
```

- `async` menandai fungsi asinkron. Tipe kembaliannya dibungkus `Future`: `return 'Ayu'` menjadi `Future<String>`.
- `await` menunggu Future selesai lalu mengambil nilainya. `await` hanya boleh dipakai di dalam fungsi `async`.
- Selama menunggu, event loop bebas mengerjakan hal lain, misalnya menggambar ulang layar di Flutter.

then() vs await

```dart
Future<int> ambilJumlahSks() {
  return Future.delayed(const Duration(milliseconds: 300), () => 21);
}

Future<void> main() async {
  // Gaya 1: callback then()
  await ambilJumlahSks().then((sks) => print('then(): $sks SKS'));

  // Gaya 2: async/await (lebih mudah dibaca)
  final sks = await ambilJumlahSks();
  print('await: $sks SKS');
  // Output:
  // then(): 21 SKS
  // await: 21 SKS
}
```

Keduanya setara. `async/await` lebih disarankan karena kode terbaca berurutan dari atas ke bawah, terutama jika ada beberapa langkah yang saling bergantung.

Menangani error pada kode async

```dart
import 'dart:async';

Future<double> ambilIpk(String nim) async {
  await Future.delayed(const Duration(milliseconds: 300));
  if (!nim.startsWith('H1D')) {
    throw Exception('NIM $nim tidak ditemukan');
  }
  return 3.67;
}

Future<void> main() async {
  for (final nim in ['H1D023001', 'X999']) {
    try {
      final ipk = await ambilIpk(nim);
      print('$nim: IPK $ipk');
    } catch (e) {
      print('Gagal: $e');
    }
  }

  // Membatasi waktu tunggu
  try {
    await ambilIpk('H1D023001').timeout(const Duration(milliseconds: 100));
  } on TimeoutException {
    print('Server terlalu lama merespons');
  }
  // Output:
  // H1D023001: IPK 3.67
  // Gagal: Exception: NIM X999 tidak ditemukan
  // Server terlalu lama merespons
}
```

Dengan `await`, error dari Future bisa ditangkap memakai `try/catch` biasa, persis seperti kode sinkron. `.timeout()` melempar `TimeoutException` (dari `dart:async`) jika Future tidak selesai dalam batas waktu.

Menjalankan beberapa Future secara paralel

Jika beberapa permintaan tidak saling bergantung, jangan di-`await` satu per satu. Jalankan bersamaan dengan `Future.wait`.

```
Future<int> ambilNilai(String matkul) async {
  await Future.delayed(const Duration(milliseconds: 400));
  return switch (matkul) {
    'Mobile' => 88,
    'Web' => 79,
    _ => 70,
  };
}

Future<void> main() async {
  final nilai = await Future.wait([
    ambilNilai('Mobile'),
    ambilNilai('Web'),
    ambilNilai('AI'),
  ]);
  print(nilai); // Output: [88, 79, 70]
}
```

Ketiga permintaan berjalan bersamaan sehingga total waktunya sekitar 0,4 detik, bukan 1,2 detik. Urutan hasil selalu sama dengan urutan Future di dalam list.

Jika tipe hasilnya berbeda-beda, Dart 3 menyediakan `.wait` pada record:

```
Future<String> ambilProfil() async {
  await Future.delayed(const Duration(milliseconds: 500));
  return 'Profil Ayu';
}

Future<List<String>> ambilJadwal() async {
  await Future.delayed(const Duration(milliseconds: 500));
  return ['Mobile', 'Web'];
}

Future<int> ambilNotifikasi() async {
  await Future.delayed(const Duration(milliseconds: 500));
  return 3;
}

Future<void> main() async {
  final stopwatch = Stopwatch()..start();

  // Berurutan: total sekitar 1,5 detik
  final profil = await ambilProfil();
  final jadwal = await ambilJadwal();
  final notif = await ambilNotifikasi();
  print('Berurutan: $profil, $jadwal, $notif notifikasi');
```

```
  print('Lebih dari 1,4 detik? ${stopwatch.elapsedMilliseconds > 1400}');

  stopwatch.reset();

  // Paralel dengan record .wait: total sekitar 0,5 detik
  final (p, j, n) = await (ambilProfil(), ambilJadwal(), ambilNotifikasi()).wait;
  print('Paralel: $p, $j, $n notifikasi');
  print('Kurang dari 1 detik? ${stopwatch.elapsedMilliseconds < 1000}');
  // Output:
  // Berurutan: Profil Ayu, [Mobile, Web], 3 notifikasi
  // Lebih dari 1,4 detik? true
  // Paralel: Profil Ayu, [Mobile, Web], 3 notifikasi
  // Kurang dari 1 detik? true
}
```

Stream: rangkaian data dari waktu ke waktu

Jika `Future` menghasilkan **satu** nilai, `Stream` menghasilkan **banyak** nilai yang datang bergantian, misalnya pesan chat, posisi GPS, atau perubahan data realtime dari Supabase (Modul 12).

```
Future<void> main() async {
  final stream = Stream.fromIterable(['Ayu', 'Bima', 'Citra']);

  await for (final nama in stream) {
    print('Hadir: $nama');
  }
  print('Absensi selesai');
  // Output:
  // Hadir: Ayu
  // Hadir: Bima
  // Hadir: Citra
  // Absensi selesai
}
```

`await for` menunggu setiap data yang datang, lalu keluar dari perulangan ketika stream selesai.

Membuat Stream dengan `async*` dan `yield`

```
Stream<int> hitungMundur(int dari) async* {
  for (var i = dari; i >= 1; i--) {
    await Future.delayed(const Duration(milliseconds: 300));
    yield i; // kirim satu nilai ke pendengar
  }
}

Future<void> main() async {
  await for (final detik in hitungMundur(3)) {
    print('$detik...');
  }
  print('Ujian dimulai!');

  // Stream juga punya where, map, take, toList seperti List
  final hasil = await Stream.fromIterable([1, 2, 3, 4, 5, 6])
      .where((n) => n.isEven)
      .map((n) => n * 10)
      .toList();
```

```
  print(hasil);
  // Output:
  // 3...
  // 2...
  // 1...
  // Ujian dimulai!
  // [20, 40, 60]
}
```

Fungsi `async*` mengembalikan `Stream`. Setiap `yield` mengirim satu nilai tanpa mengakhiri fungsi, berbeda dengan `return` yang mengakhiri fungsi.

listen() dan StreamController

`StreamController` dipakai ketika kita ingin "mengirim" data ke stream secara manual, misalnya setiap kali isi keranjang belanja berubah.

```
import 'dart:async';

class Keranjang {
  final _controller = StreamController<int>();
  int _jumlah = 0;

  Stream<int> get jumlahStream => _controller.stream;

  void tambah() {
    _jumlah++;
    _controller.add(_jumlah); // kirim data baru ke stream
  }

  Future<void> tutup() => _controller.close();
}

Future<void> main() async {
  final keranjang = Keranjang();

  keranjang.jumlahStream.listen(
    (jumlah) => print('Isi keranjang: $jumlah'),
    onDone: () => print('Stream ditutup'),
  );

  keranjang.tambah();
  keranjang.tambah();
  keranjang.tambah();
  await keranjang.tutup(); // tunggu sampai semua data terkirim
  // Output:
  // Isi keranjang: 1
  // Isi keranjang: 2
  // Isi keranjang: 3
  // Stream ditutup
}
```

`listen()` mendaftarkan fungsi yang dipanggil setiap ada data baru. Secara default, stream hanya boleh punya satu pendengar; gunakan `StreamController.broadcast()` jika butuh banyak

pendengar. Di Flutter, `StreamBuilder` melakukan `listen` secara otomatis dan membangun ulang widget setiap kali data baru datang.

	`Future<T>`	`Stream<T>`
Jumlah nilai	Satu kali	Nol sampai banyak
Cara menunggu	`await`	`await for` / `listen()`
Membuat fungsi	`async` + `return`	`async*` + `yield`
Widget Flutter	`FutureBuilder`	`StreamBuilder`
Contoh	Login, GET data API	Chat, lokasi GPS, data realtime

> ⚠️ **Kesalahan umum**
>
> - Lupa `await` : variabel berisi `Instance of 'Future<String>'` , bukan datanya.
> - Memakai `await` di fungsi yang tidak ditandai `async` , sehingga terjadi error compile.
> - Meng- `await` satu per satu permintaan yang sebenarnya bisa paralel. Gunakan `Future.wait` atau record `.wait` .
> - Lupa menutup `StreamController` atau membatalkan `StreamSubscription` (di Flutter, lakukan di `dispose()`), sehingga terjadi kebocoran memori.

12. Studi Kasus: Sistem Nilai & IPK Mahasiswa

Saatnya menggabungkan semua yang sudah dipelajari menjadi satu program utuh: **Sistem Nilai dan IPK Mahasiswa**. Program ini mengambil data KHS dari "server" (simulasi), memvalidasi data yang rusak, lalu menghitung IPK, predikat, dan mata kuliah yang perlu diulang.

Spesifikasi

- Setiap mata kuliah punya kode, nama, SKS (1 sampai 6), dan nilai angka (0 sampai 100).
- Nilai angka dikonversi ke nilai huruf dan bobot: A=4.0, AB=3.5, B=3.0, BC=2.5, C=2.0, D=1.0, E=0.0. Nilai di bawah C wajib diulang.
- IPK = jumlah (SKS × bobot) dibagi total SKS.
- Predikat: ≥ 3.51 Dengan Pujian, ≥ 3.01 Sangat Memuaskan, ≥ 2.76 Memuaskan, selain itu Cukup.
- Data dari server bisa rusak. Data yang rusak dilewati dan dilaporkan sebagai peringatan, tanpa membuat program berhenti.

Kode lengkap

Simpan sebagai satu file lalu jalankan dengan `dart run sistem_nilai.dart`. Komentar bernomor menandai bagian-bagian program.

sistem_nilai.dart

```
// ===== 1. ENUM: skala nilai huruf (enhanced enum) =====
enum NilaiHuruf {
  a('A', 4.0, 85),
  ab('AB', 3.5, 80),
  b('B', 3.0, 70),
  bc('BC', 2.5, 65),
  c('C', 2.0, 60),
  d('D', 1.0, 50),
  e('E', 0.0, 0);

  const NilaiHuruf(this.label, this.bobot, this.batasBawah);

  final String label;
  final double bobot;
  final int batasBawah;

  bool get lulus => bobot >= 2.0;

  static NilaiHuruf dariAngka(double nilai) =>
      values.firstWhere((h) => nilai >= h.batasBawah);
}

// ===== 2. EXCEPTION kustom =====
class DataNilaiException implements Exception {
  final String pesan;
  DataNilaiException(this.pesan);
```

```dart
  @override
  String toString() => 'DataNilaiException: $pesan';
}

// ===== 3. CLASS MataKuliah (constructor, validasi, factory, getter) =====
class MataKuliah {
  final String kode;
  final String nama;
  final int sks;
  final double nilaiAngka;

  MataKuliah({
    required this.kode,
    required this.nama,
    required this.sks,
    required this.nilaiAngka,
  }) {
    if (sks < 1 || sks > 6) {
      throw DataNilaiException('SKS $kode harus 1-6');
    }
    if (nilaiAngka < 0 || nilaiAngka > 100) {
      throw DataNilaiException('Nilai $kode di luar rentang 0-100');
    }
  }

  factory MataKuliah.fromJson(Map<String, dynamic> json) {
    if (json
        case {
          'kode': String kode,
          'nama': String nama,
          'sks': int sks,
          'nilai': num nilai,
        }) {
      return MataKuliah(
        kode: kode,
        nama: nama,
        sks: sks,
        nilaiAngka: nilai.toDouble(),
      );
    }
    throw DataNilaiException('Format data ${json['kode']} tidak sesuai');
  }

  NilaiHuruf get huruf => NilaiHuruf.dariAngka(nilaiAngka);
  double get mutu => sks * huruf.bobot;
}

// ===== 4. EXTENSION =====
extension FormatAngka on double {
  String get duaDesimal => toStringAsFixed(2);
}

// ===== 5. SEALED CLASS: hasil pemuatan data =====
sealed class HasilMuat {}

class MuatBerhasil extends HasilMuat {
  final List<MataKuliah> data;
  final List<String> peringatan;
```

```dart
  MuatBerhasil(this.data, this.peringatan);
}

class MuatGagal extends HasilMuat {
  final String pesan;
  MuatGagal(this.pesan);
}

// ===== 6. CLASS Mahasiswa (private, getter, koleksi, record) =====
class Mahasiswa {
  final String nim;
  final String nama;
  final List<MataKuliah> _khs = [];

  Mahasiswa(this.nim, this.nama);

  List<MataKuliah> get khs => List.unmodifiable(_khs);

  void tambahSemua(List<MataKuliah> daftar) => _khs.addAll(daftar);

  int get totalSks => _khs.fold(0, (total, mk) => total + mk.sks);

  double get ipk {
    if (totalSks == 0) return 0;
    final totalMutu = _khs.fold(0.0, (total, mk) => total + mk.mutu);
    return totalMutu / totalSks;
  }

  String get predikat => switch (ipk) {
        >= 3.51 => 'Dengan Pujian',
        >= 3.01 => 'Sangat Memuaskan',
        >= 2.76 => 'Memuaskan',
        _ => 'Cukup',
      };

  List<MataKuliah> get perluMengulang =>
      _khs.where((mk) => !mk.huruf.lulus).toList();

  ({MataKuliah terbaik, MataKuliah terendah}) get ringkasan {
    final urut = [..._khs]..sort((a, b) => b.nilaiAngka.compareTo(a.nilaiAngka));
    return (terbaik: urut.first, terendah: urut.last);
  }
}

// ===== 7. ASYNC: simulasi mengambil data dari server =====
Future<HasilMuat> muatKhsDariServer(String nim) async {
  await Future.delayed(const Duration(milliseconds: 500));
  if (nim.isEmpty) return MuatGagal('NIM kosong');

  final dataJson = <Map<String, dynamic>>[
    {'kode': 'IF201', 'nama': 'Pemrograman Mobile', 'sks': 3, 'nilai': 88},
    {'kode': 'IF202', 'nama': 'Basis Data', 'sks': 3, 'nilai': 76.5},
    {'kode': 'IF203', 'nama': 'Jaringan Komputer', 'sks': 2, 'nilai': 64},
    {'kode': 'IF204', 'nama': 'Kecerdasan Buatan', 'sks': 3, 'nilai': 81},
    {'kode': 'IF205', 'nama': 'Etika Profesi', 'sks': 2, 'nilai': 55},
    {'kode': 'IF206', 'nama': 'Statistika', 'sks': 3, 'nilai': 'A'}, // rusak
    {'kode': 'IF207', 'nama': 'Kerja Praktik', 'sks': 8, 'nilai': 90}, // rusak
  ];
```

```dart
  final data = <MataKuliah>[];
  final peringatan = <String>[];
  for (final json in dataJson) {
    try {
      data.add(MataKuliah.fromJson(json));
    } on DataNilaiException catch (e) {
      peringatan.add(e.pesan);
    }
  }
  return MuatBerhasil(data, peringatan);
}

// ===== 8. MAIN =====
Future<void> main() async {
  final mhs = Mahasiswa('H1D023001', 'Ayu Lestari');
  print('Memuat KHS ${mhs.nama}...');

  final hasil = await muatKhsDariServer(mhs.nim);
  switch (hasil) {
    case MuatGagal(:final pesan):
      print('Gagal memuat: $pesan');
      return;
    case MuatBerhasil(:final data, :final peringatan):
      mhs.tambahSemua(data);
      for (final p in peringatan) {
        print('Peringatan: $p');
      }
  }

  print('');
  print('KHS ${mhs.nim} - ${mhs.nama}');
  for (final (i, mk) in mhs.khs.indexed) {
    print('${i + 1}. ${mk.kode} ${mk.nama.padRight(20)}'
        '${mk.sks} SKS  ${mk.nilaiAngka.toString().padLeft(4)}  ${mk.huruf.label}');
  }

  final (:terbaik, :terendah) = mhs.ringkasan;
  print('');
  print('Total SKS   : ${mhs.totalSks}');
  print('IPK         : ${mhs.ipk.duaDesimal}');
  print('Predikat    : ${mhs.predikat}');
  print('Terbaik     : ${terbaik.nama} (${terbaik.huruf.label})');
  print('Terendah    : ${terendah.nama} (${terendah.huruf.label})');
  print('Mengulang   : ${mhs.perluMengulang.map((mk) => mk.nama).join(', ')}');
}
```

Output

```
Memuat KHS Ayu Lestari...
Peringatan: Format data IF206 tidak sesuai
Peringatan: SKS IF207 harus 1-6

KHS H1D023001 - Ayu Lestari
1. IF201 Pemrograman Mobile  3 SKS  88.0  A
2. IF202 Basis Data          3 SKS  76.5  B
3. IF203 Jaringan Komputer   2 SKS  64.0  C
4. IF204 Kecerdasan Buatan   3 SKS  81.0  AB
5. IF205 Etika Profesi       2 SKS  55.0  D
```

```
Total SKS   : 13
IPK         : 2.88
Predikat    : Memuaskan
Terbaik     : Pemrograman Mobile (A)
Terendah    : Etika Profesi (D)
Mengulang   : Etika Profesi
```

Penjelasan per bagian

Bagian	Konsep yang dipakai	Section
1. `NilaiHuruf`	Enhanced enum dengan field, getter `lulus`, dan method static `dariAngka` memakai `firstWhere`	8, 6
2. `DataNilaiException`	Exception kustom dengan `implements Exception` dan override `toString()`	10
3. `MataKuliah`	Named parameter `required`, validasi di body constructor dengan `throw`, factory `fromJson` dengan `if-case` pattern, getter terhitung	5, 8, 9
4. `FormatAngka`	Extension method pada `double`	8
5. `HasilMuat`	Sealed class dan exhaustive switch di `main()` dengan object pattern `:final pesan`	9
6. `Mahasiswa`	Field private `_khs`, `List.unmodifiable`, `fold`, `where`, switch expression dengan relational pattern, getter yang mengembalikan record	6, 8, 9
7. `muatKhsDariServer`	`async` / `await`, `Future.delayed`, `try` / `on ... catch` di dalam perulangan	10, 11
8. `main()`	`Future<void> main() async`, destructuring `.indexed` dan record, interpolasi, `padRight` / `padLeft`, `map` + `join`	2, 4, 6, 9

Beberapa hal yang perlu diperhatikan:

- **Data rusak tidak membuat program berhenti.** `IF206` ditolak oleh pattern di `fromJson` karena nilainya String, sedangkan `IF207` lolos pattern tetapi ditolak validasi SKS di constructor. Keduanya melempar `DataNilaiException` yang ditangkap di dalam perulangan, sehingga data lain tetap diproses.
- **Perhitungan IPK**: total mutu = 3×4.0 + 3×3.0 + 2×2.0 + 3×3.5 + 2×1.0 = 37.5. Dibagi 13 SKS menghasilkan 2.8846..., yang dibulatkan menjadi 2.88 oleh extension `duaDesimal`.

- `fold(0.0, ...)` sengaja diberi nilai awal `0.0` (double), karena `mutu` bertipe double. Jika ditulis `0`, Dart menganggap akumulatornya `int` dan terjadi error tipe.
- **Getter** `khs` mengembalikan list yang tidak bisa diubah. Kode di luar class tidak bisa menambah mata kuliah tanpa lewat method `tambahSemua`.

Bonus: progres sinkronisasi dengan Stream

Fitur tambahan yang sering muncul di aplikasi: menampilkan persentase progres saat data dikirim ke server. Ini cocok dibuat dengan `async*`.

```
Stream<int> sinkronkanKrs(int jumlahMatkul) async* {
  for (var i = 1; i <= jumlahMatkul; i++) {
    await Future.delayed(const Duration(milliseconds: 200)); // kirim satu matkul
    yield i * 100 ~/ jumlahMatkul; // persentase selesai
  }
}

Future<void> main() async {
  await for (final persen in sinkronkanKrs(4)) {
    print('Menyinkronkan... $persen%');
  }
  print('Data KRS Ayu Lestari berhasil disinkronkan');
  // Output:
  // Menyinkronkan... 25%
  // Menyinkronkan... 50%
  // Menyinkronkan... 75%
  // Menyinkronkan... 100%
  // Data KRS Ayu Lestari berhasil disinkronkan
}
```

Di Flutter, stream seperti ini bisa dihubungkan ke `StreamBuilder` untuk menggerakkan `LinearProgressIndicator`.

> 💡 **Latihan pengembangan (mandiri)**
>
> 1. Tambahkan field `semester` pada `MataKuliah`, lalu hitung **IPS** (indeks prestasi per semester) memakai `where`.
> 2. Jika satu mata kuliah diambil dua kali, gunakan nilai terbaik saja saat menghitung IPK. Petunjuk: kelompokkan berdasarkan `kode` dengan `Map`.
> 3. Buat `muatKhsDariServer` mengembalikan `MuatGagal` secara acak (gunakan `Random` dari `dart:math`), lalu pastikan pesan gagal tampil dengan benar.
> 4. Tambahkan method `toJson()` pada `MataKuliah` dan cetak seluruh KHS dalam format JSON memakai `jsonEncode` dari `dart:convert`.

Tugas Praktikum: Praktikum 2: Sistem Pengelolaan Nilai & IPK Mahasiswa dengan Dart 3

Sasaran Praktikum

Menerapkan seluruh dasar Dart (variabel, control flow, fungsi, koleksi, null safety, OOP, fitur Dart 3, error handling, dan async) secara bertahap dalam program console pengelolaan nilai mahasiswa.

Langkah Pengerjaan

1. **Persiapan.** Buat proyek dengan `dart create -t console praktikum_dart`. Kerjakan setiap tugas di file terpisah di folder `bin/` (misalnya `bin/tugas1.dart`) dan jalankan dengan `dart run bin/tugas1.dart`. Pastikan `dart analyze` tidak menampilkan error.
2. **Tugas 1 - Variabel & Control Flow (10%).** Simpan daftar nilai `[92, 78, 65, 54, 81, 47]` dalam `List<int>`. Dengan perulangan `for-in` dan *switch expression* ber-relational pattern, cetak setiap nilai beserta huruf mutunya (A/AB/B/BC/C/D/E). Gunakan `continue` untuk melewati nilai negatif jika ada.
3. **Tugas 2 - Fungsi & Koleksi (15%).** Buat fungsi `hitungStatistik(List<int> nilai)` yang mengembalikan *record* `({double rataRata, int tertinggi, int terendah, int jumlahLulus})` memakai `fold`, `reduce`, dan `where`. Buat juga fungsi dengan named parameter `cetakLaporan({required String judul, required List<int> nilai, int batasLulus = 60})`.
4. **Tugas 3 - OOP (25%).** Buat *enhanced enum* `NilaiHuruf` (label, bobot, batas bawah), class `MataKuliah` (kode, nama, sks, nilaiAngka) dengan getter `huruf` dan `mutu`, serta class `Mahasiswa` dengan field private `_khs`, getter `totalSks`, `ipk`, dan `predikat`. Tambahkan `factory MataKuliah.fromJson(Map<String, dynamic> json)` dan override `toString()`.
5. **Tugas 4 - Null Safety & Error Handling (15%).** Buat exception kustom `DataNilaiException`. Lempar exception ini jika SKS di luar 1-6 atau nilai di luar 0-100. Baca input nilai dari List String (misalnya `['85', 'delapan', '70']`) memakai `int.tryParse` dan operator `??`, lalu laporkan data yang tidak valid tanpa menghentikan program.
6. **Tugas 5 - Fitur Dart 3 (15%).** Buat `sealed class HasilMuat` dengan turunan `MuatBerhasil` dan `MuatGagal`. Tangani keduanya dengan *exhaustive switch* dan object pattern. Gunakan `if-case` di `fromJson` untuk memvalidasi bentuk JSON, serta `.indexed` dengan destructuring saat mencetak KHS bernomor.
7. **Tugas 6 - Asynchronous (20%).** Buat fungsi `Future<HasilMuat> muatKhsDariServer(String nim)` dengan `Future.delayed`. Ambil data dua semester secara paralel memakai `Future.wait` atau record `.wait`. Buat juga `Stream<int> simpanKeServer()` dengan `async*` yang mengirim

persentase progres, lalu cetak 'Data KRS Mahasiswa [Nama] berhasil disinkronkan' setelah stream selesai.

8. **Pengumpulan.** Kumpulkan folder proyek (tanpa folder `.dart_tool`) dan tangkapan layar output terminal setiap tugas. Sertakan komentar singkat di setiap file yang menjelaskan konsep yang dipakai.

Output yang Diharapkan

Enam program console yang berjalan tanpa error dan lolos dart analyze: konversi nilai ke huruf mutu, laporan statistik nilai, KHS bernomor lengkap dengan total SKS, IPK dua desimal, dan predikat, peringatan untuk data yang tidak valid tanpa program berhenti, serta simulasi pemuatan data paralel dan progres sinkronisasi asinkron.

Rubrik Penilaian

Tugas 1 Variabel & Control Flow (10%), Tugas 2 Fungsi & Koleksi (15%), Tugas 3 OOP: enum, class, constructor, getter, factory (25%), Tugas 4 Null Safety & Error Handling (15%), Tugas 5 Records/Patterns/Sealed Class (15%), Tugas 6 Future, Future.wait & Stream (20%). Pengurangan nilai jika dart analyze menampilkan error atau output tidak sesuai.

Kuis Pemahaman

1. Apa perbedaan utama antara final dan const di Dart?

A. final untuk angka, const untuk String

B. final diisi sekali dan nilainya boleh ditentukan saat runtime, sedangkan const harus sudah diketahui saat compile-time

C. const boleh diubah, final tidak boleh

D. Tidak ada perbedaan, keduanya sama persis

2. Apa hasil dari print(17 ~/ 5) dan print(17 / 5) di Dart?

A. 3 dan 3

B. 3.4 dan 3

C. 3 dan 3.4

D. 2 dan 3.4

3. Apa fungsi operator ?? (null-coalescing) pada Dart?

A. Memaksa compiler mengabaikan null safety

B. Memberikan nilai cadangan jika ekspresi di sisi kirinya bernilai null

C. Mengecek apakah dua objek identik di memori

D. Menghapus nilai null dari List secara otomatis

4. Perhatikan deklarasi void daftar({required String nim, String prodi = 'Informatika'}). Pemanggilan mana yang VALID?

A. daftar('H1D023001')

B. daftar(prodi: 'Elektro')

C. daftar(nim: 'H1D023001')

D. daftar()

5. Apa output dari kode berikut? var n = [78, 92, 55]; print(n.where((x) ⇒ x >= 60).map((x) ⇒ x + 5).toList());

A. [83, 97, 60]

B. [83, 97]

C. [78, 92]

D. (83, 97)

6. Kapan sebaiknya kita memakai operator ! (null assertion) pada variabel nullable?

A. Setiap kali compiler menampilkan error null, agar error cepat hilang

B. Hanya jika kita benar-benar yakin nilainya tidak null, karena jika ternyata null aplikasi akan crash saat runtime

C. Untuk mengubah variabel menjadi nullable

D. Untuk memberi nilai default ketika variabel null

7. Apa perbedaan extends, implements, dan with pada class Dart?

A. Ketiganya sama, hanya beda gaya penulisan

B. extends mewarisi isi dari satu induk, implements berjanji menyediakan semua method interface tanpa mewarisi isinya, with menempelkan mixin beserta isinya

C. extends boleh banyak induk, implements hanya satu

D. with hanya bisa dipakai pada abstract class

8. Apa keuntungan utama sealed class ketika dipakai bersama switch expression?

A. Program berjalan lebih cepat

B. Objek sealed class tidak bisa bernilai null

C. Compiler mengetahui semua turunannya sehingga memberi error jika ada kasus yang belum ditangani (exhaustive checking)

D. Sealed class bisa dibuat objeknya secara langsung

9. Apa yang terjadi pada kode berikut? try { int.parse('12a'); print('A'); } on FormatException { print('B'); } finally { print('C'); }

A. Mencetak A lalu C

B. Mencetak B lalu C

C. Mencetak A, B, lalu C

D. Program crash tanpa mencetak apa pun

10. Tiga permintaan API yang tidak saling bergantung masing-masing butuh 1 detik. Cara paling efisien untuk menunggu ketiganya adalah...

A. Meng-await ketiganya satu per satu secara berurutan (total sekitar 3 detik)

B. Memakai Future.wait (atau record .wait) agar berjalan paralel (total sekitar 1 detik)

C. Memakai Stream.fromIterable

D. Tidak memakai await sama sekali lalu langsung membaca hasilnya

Kunci Jawaban & Pembahasan

1. B — final hanya bisa diisi sekali tetapi nilainya boleh berasal dari runtime (misalnya DateTime.now()). const harus berupa nilai compile-time, sehingga objek const yang isinya sama bisa dipakai bersama dan dihemat memorinya, seperti const widget di Flutter.

2. C — Operator ~/ melakukan pembagian bulat sehingga hasilnya 3, sedangkan / selalu menghasilkan double, yaitu 3.4. Sisa baginya (17 % 5) adalah 2.

3. B — a ?? b memakai a jika tidak null, dan memakai b jika a bernilai null. Contoh: email ?? 'Belum diisi'.

4. C — nim adalah named parameter yang required, jadi wajib ditulis dengan namanya (nim: ...). prodi punya nilai default sehingga boleh dihilangkan. Named parameter tidak bisa diisi berdasarkan posisi.

5. B — where menyaring nilai >= 60 sehingga tersisa [78, 92], lalu map menambah 5 menjadi [83, 97]. toList() mengubah Iterable menjadi List sehingga dicetak dengan kurung siku.

6. B — Operator ! mematikan perlindungan null safety. Jika nilainya ternyata null, muncul error 'Null check operator used on a null value'. Pilihan yang lebih aman biasanya pengecekan if, ?. atau ??.

7. B — Sebuah class hanya bisa extends satu class, tetapi boleh implements banyak interface (semua method wajib ditulis ulang) dan with banyak mixin (isinya ikut terbawa).

8. C — Turunan sealed class hanya boleh ditulis di file yang sama, sehingga compiler tahu daftar lengkapnya. Switch tanpa cabang _ akan ditolak jika ada turunan yang terlewat, cocok untuk status seperti Memuat, Sukses, dan Gagal.

9. B — int.parse('12a') melempar FormatException sehingga print('A') dilewati. Blok on FormatException mencetak B, dan finally selalu dijalankan sehingga mencetak C.

10. B — Future.wait([...]) atau (f1, f2, f3).wait menjalankan semua Future bersamaan dan menunggu hingga semuanya selesai, sehingga total waktu kira-kira sama dengan permintaan yang paling lama.